

M3Admin CLI User Guide

M3AD-01

July 2026

Version 2.0

Table of Contents

- [Preface](#)
- 1. [General Information](#)
 - [About M3 Admin](#)
 - [Key Features](#)
 - [Architecture Overview](#)
 - [Integration with Modular Ecosystem](#)
 - [Prerequisites](#)
- 2. [Installation and Configuration](#)
 - [Prerequisites Verification](#)
 - [Installation Steps](#)
 - [Initial Configuration](#)
 - [Mode-Specific Configuration](#)
 - [Configuration Parameters Reference](#)
 - [Log Configuration](#)
 - [Verify Installation and Configuration](#)
- 3. [Permissions and Access](#)
 - [Permission Model](#)
 - [AWS Credentials Resolving](#)
- 4. [Command Line Usage](#)
 - [Command Usage](#)
 - [Output Formats](#)
 - [Autocompletion](#)
- 5. [Cloud Provider Management](#)
 - [Supported Cloud Providers](#)
 - [AWS Cloud Management](#)
 - [OpenStack Management](#)
 - [Cross-Cloud Operations](#)
 - [Application Credentials Management](#)
 - [Best Practices](#)
- 6. [Audit Service](#)
 - [Audit Overview](#)
 - [Audit Configuration](#)
 - [Audit Table Structure](#)
 - [Querying Audit Events](#)
 - [Audit Examples](#)
 - [Secured Parameters](#)
 - [Audit Best Practices](#)
 - [Troubleshooting Audit](#)
 - [Audit Event Decorator](#)
- 7. [Advanced Configuration](#)
 - [Project Conventions](#)
 - [Multiple Database Connections \(MongoDB\)](#)
 - [Custom Mode Combinations](#)
 - [Environment-Specific Configurations](#)
 - [Security Hardening](#)
 - [Performance Tuning](#)
- 8. [Project Information](#)
- 9. [Annexes](#)

- [Annex 1: Common Use Cases](#)
- [Annex 2: Troubleshooting Guide](#)

Preface

About This Guide

This guide describes the installation, management and usage of M3 Admin – a comprehensive command-line administration tool for the Maestro environment. It provides centralized management capabilities for multi-cloud infrastructure, enabling administrators to manage applications, parents, tenants, credentials, and resources across AWS, Azure, Google Cloud Platform, and private cloud environments.

Target Audience

The guide is designed for Maestro administrative users who have the responsibility to perform configuration and setup tasks, maintenance works and assisting users with matters beyond the self-service scope.

Related Documents

- For the detailed information on the CLI commands, their parameters and usage, refer to M3 Admin Reference Guide.
- For the information about Maestro tool usage via a command line interface, see M3 CLI User Guide.

1. General Information

About M3 Admin

M3 Admin is a comprehensive command-line administration tool for the Maestro environment. It provides centralized management capabilities for multi-cloud infrastructure, enabling administrators to manage applications, parents, tenants, credentials, and resources across AWS, Azure, Google Cloud Platform, and private cloud environments.

M3 Admin is part of the Modular ecosystem within Maestro.

Key Features

- **Multi-Cloud Management:** Unified interface for AWS, Azure, GCP, and private clouds
- **Tenant Lifecycle Management:** Activate, configure, and deactivate cloud tenants (accounts)
- **Credential Management:** Secure storage and rotation of cloud credentials
- **Application Management:** Centralized application (access metadata) and parent (relationship between Parents and Tenants) entity configuration
- **Security Integration:** Integration with SRE (Syndicate Rule Engine), Qualys, and other security platforms
- **Billing Configuration:** Configure billing sources and cost allocation
- **Audit Capabilities:** Comprehensive audit logging and reporting
- **Permission-Based Access:** Fine-grained policy-based access control
- **Multiple Deployment Modes:** SaaS, On-Premises, and Custom (Hybrid) modes

Note: The detailed information about Application -> Parent -> Tenant model can be found in the [modular-sdk](#)

Architecture Overview

M3 Admin can operate in three distinct modes:

Mode	Database	Blob Storage	Secrets Management	Dependencies
SaaS Mode (Default)	AWS DynamoDB	AWS S3	AWS SSM	Requires AWS credentials
On-Premises Mode	MongoDB	MinIO	Vault	No cloud provider dependencies
Custom Mode	DynamoDB or MongoDB	S3 or MinIO	SSM or Vault	Maximum flexibility for hybrid deployments

Integration with Modular Ecosystem as Part of Maestro

M3 Admin integrates with:

- **Modular-API:** Can be installed as a Modular-API module
- **Modular-CLI:** Can be accessed through Modular-CLI interface
- **Modular-SDK:** Uses Modular-SDK for backend communication
- **M3 Server:** Communicates via RabbitMQ (or directly to the server) for long-running operations
- **Database Services:** DynamoDB or MongoDB for data storage

Prerequisites

Before installing M3 Admin, ensure you have:

- **Python 3.14** installed
- **pip** package manager
- **Virtual environment** (highly recommended)

- **Modular-SDK** (automatically installed as dependency)
- **Access credentials** for target cloud providers
- **RabbitMQ connection** (for server communication)
- **Database access** (DynamoDB or MongoDB depending on mode)

[Content ↑](#)

2. Installation and Configuration

Prerequisites Verification

Download and install Python 3.14:

- [Python for Windows](#)
- [Python for Linux](#)
- [Python for Mac](#)

Verify installation:

```
python --version
# Expected: Python 3.14
```

```
pip --version
# Expected: pip 21.x or higher
```

IMPORTANT: Using a virtual environment is highly recommended to prevent dependency conflicts. For more detail, see the [Virtual Environments Creation Guide](#).

Installation Steps

1. Create a Virtual Environment

On Linux/Mac:

```
python3.14 -m venv m3admin_venv
source m3admin_venv/bin/activate
```

On Windows (PowerShell):

```
python -m venv m3admin_venv
.\m3admin_venv\Scripts\Activate.ps1
```

On Windows (Command Prompt):

```
python -m venv m3admin_venv
.\m3admin_venv\Scripts\activate.bat
```

2. Clone the M3 Admin Repository

```
git clone https://git.epam.com/epmc-eoos/m3admin
cd m3admin
```

3. Install M3 Admin

Standalone Installation:

```
# Install from source
pip install ./src

# Verify installation
m3admin --version
```

This automatically installs: * Core M3 Admin package * Modular-SDK (from PyPI) * All dependencies from `pyproject.toml`

Installation as Modular-API Module:

See Modular-API documentation for module installation instructions.

Initial Configuration

M3 Admin requires configuration before first use. The configuration varies based on the deployment mode.

Make Environment Variables Permanent:

1. Create Configuration Directory

On Windows:

```
# Create configuration directory
New-Item -Path "m3-config" -ItemType Directory

# Set environment variables
$env:M3ADMIN_CONF="C:\path\to\m3admin\m3-config"
$env:M3_ADMIN_HOME="C:\path\to\m3admin"
```

Linux/Mac (add to ~/.bashrc or ~/.zshrc):

```
# Create configuration directory

mkdir -p m3-config

# Set environment variables
export M3ADMIN_CONF=/path/to/m3admin/m3-config
export M3_ADMIN_HOME=/path/to/m3admin

echo 'export M3ADMIN_CONF=/path/to/m3admin/m3-config' >> ~/.bashrc
echo 'export M3_ADMIN_HOME=/path/to/m3admin' >> ~/.bashrc
source ~/.bashrc
```

Windows (System Environment Variables):

```
[System.Environment]::SetEnvironmentVariable('M3ADMIN_CONF', 'C:\path\to\m3admin\m3-config',
'User')
[System.Environment]::SetEnvironmentVariable('M3_ADMIN_HOME', 'C:\path\to\m3admin', 'User')
```

2. Create Policy File

Create m3admin_policy.json in the m3-config directory:

```
cat > $M3ADMIN_CONF/m3admin_policy.json << 'EOF'
[
  {
    "Effect": "Allow",
    "Group": "*",
    "Resources": ["*"]
  }
]
EOF
```

NOTE: This policy grants full access. See [Permission Model](#) for fine-grained access control.

3. Create a Configuration File

Create m3admin.conf in the m3-config directory based on your deployment mode.

Mode-Specific Configuration

The instructions below provide the instruction on M3 Admin configuration depending on the installation mode. The [Configuration Parameters Reference](#) at the end of the section provides parameter details.

SaaS Mode Configuration

Create `$M3ADMIN_CONF/m3admin.conf` with the following content:

```
# =====
# M3 Admin Configuration - SaaS Mode
# =====

# Mode configuration (saas is default, can be omitted)
# mode=saas

# =====
# Modular SDK Configuration
# =====
# User email for Maestro authentication
maestro_user=admin@example.com

# SDK credentials (SYSTEM key type)
sdk_access_key=your_sdk_access_key_here
sdk_secret_key=your_sdk_secret_key_here

# =====
# AWS Credentials
# =====
# AWS account ID
account_id=123456789012

# AWS access credentials
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxrFicYEXAMPLEKEY

# Optional: Session token for temporary credentials
# aws_session_token=

# AWS region for resources
region=us-east-1

# Optional: IAM role to assume
# access_role=arn:aws:iam::123456789012:role/MaestroAdminRole

# Optional: MFA device ARN
# mfa_serial=arn:aws:iam::123456789012:mfa/username

# Optional: Session duration in seconds (900-129600, default: 3600)
# aws_session_duration=3600

# =====
# S3 Configuration
# =====
# S3 bucket for reports
report_bucket=maestro-reports-bucket
```

```
# =====
# Version Check
# =====
# Check if m3admin version matches server version (True/False)
check_relevance=True

# =====
# RabbitMQ Configuration
# =====
# RabbitMQ connection URL
rabbit_connection_url=amqp://username:password@rabbitmq.example.com:5672/

# RabbitMQ queues
rabbit_api_queue=m3_api_queue
rabbit_request_queue=m3_request_queue
rabbit_response_queue=m3_response_queue

# Optional: Audit queue
# rabbit_audit_queue=m3_audit_queue

# Optional: RabbitMQ timeout in seconds (default: 60)
# rabbit_timeout=60

# Optional: RabbitMQ exchange name
# rabbit_exchange=m3_exchange

# =====
# Optional Settings
# =====
# API mode (set to True when installed as Modular-API module)
# api_mode=False

# API mode handler invocation tracing
# api_mode_handler_invocation=False

# Send audit events to M3 Server via RabbitMQ
# send_audit=False

# Store audit events locally in DynamoDB
# stand_alone_audit=True

# Log rotation (rotate at midnight)
# rotate_logs=False

# Custom logs path
# logs_path=/var/log/m3admin

# Custom policy path
# policy_path=/custom/path/to/m3admin_policy.json

# Enable Modular SDK logging
# enable_modular_sdk_log=True

# Custom audit user (useful in Docker)
# maestro_audit_user=admin
```

On-Premises Mode Configuration

Create \$M3ADMIN_CONF/m3admin.conf with the following content:

```
# =====
# M3 Admin Configuration - On-Premises Mode
# =====

# Mode configuration
mode=onprem

# =====
# Modular SDK Configuration
# =====
maestro_user=admin@example.com
sdk_access_key=your_sdk_access_key_here
sdk_secret_key=your_sdk_secret_key_here

# =====
# MongoDB Configuration (Newer Implementation)
# =====
# Multiple MongoDB connections support
# Format: {"name": "connection_uri", "name2": "connection_uri2"}
mongo_uris={"main":
"mongodb+srv://username:password@mongo.example.com/maestro?retryWrites=true", "secondary":
"mongodb+srv://username:password@mongo.example.com/modular_sdk_db?retryWrites=true"}

# Optional: Table/Collection mapping for different connections
# Format: {"connection_name": ["Table1", "Table2"]}
# All tables use "main" by default
mongo_table_mapping={"secondary": ["Customers", "Tenants", "Parents", "Applications",
"Regions"]}]

# =====
# MongoDB Configuration (Legacy Implementation)
# =====
# Use this if not using the newer mongo_uris implementation
# onprem_mongodb_uri=mongodb://username:password@localhost:27017/maestro
# onprem_mongodb_onprem_db_name=maestro

# =====
# MinIO Configuration
# =====
# MinIO connection URI
onprem_minio_uri=http://minio.example.com:9000

# MinIO bucket for reports
minio_report_bucket=maestro-reports

# =====
# Vault Configuration
# =====
# Vault connection URI
onprem_vault_uri=http://vault.example.com:8200

# =====
# Version Check
```

```
# =====
check_relevance=True

# =====
# RabbitMQ Configuration
# =====
rabbit_connection_url=amqp://username:password@rabbitmq.example.com:5672/
rabbit_api_queue=m3_api_queue
rabbit_request_queue=m3_request_queue
rabbit_response_queue=m3_response_queue

# =====
# Optional Settings
# =====
# stand_alone_audit=True
# rotate_logs=False
# enable_modular_sdk_log=True
```

Custom Mode Configuration

Create \$M3ADMIN_CONF/m3admin.conf with the following content:

```
# =====
# M3 Admin Configuration - Custom Mode
# =====

# Mode configuration
mode=custom

# Component selection
custom_mode_db=dynamodb      # Options: dynamodb, mongodb
custom_mode_blob=s3          # Options: s3, minio
custom_mode_secret=ssm        # Options: ssm, vault

# =====
# Modular SDK Configuration
# =====
maestro_user=admin@example.com
sdk_access_key=your_sdk_access_key_here
sdk_secret_key=your_sdk_secret_key_here

# =====
# Configuration for DynamoDB + S3 + SSM
# =====
# AWS credentials required for: dynamodb, s3, ssm
account_id=123456789012
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
region=us-east-1
report_bucket=maestro-reports-bucket

# =====
# Configuration for MongoDB (if custom_mode_db=mongodb)
# =====
# mongo_uris={"main": "mongodb+srv://username:password@mongo.example.com/maestro"}

# =====
# Configuration for MinIO (if custom_mode_blob=minio)
```

```
# =====
# onprem_minio_uri=http://minio.example.com:9000
# minio_report_bucket=maestro-reports

# =====
# Configuration for Vault (if custom_mode_secret=vault)
# =====
# onprem_vault_uri=http://vault.example.com:8200

# =====
# Version Check
# =====
check_relevance=True

# =====
# RabbitMQ Configuration
# =====
rabbit_connection_url=amqp://username:password@rabbitmq.example.com:5672/
rabbit_api_queue=m3_api_queue
rabbit_request_queue=m3_request_queue
rabbit_response_queue=m3_response_queue
```

Configuration Parameters Reference

Required Parameters

Parameter	Description	Example
maestro_user	User email for Maestro authentication	admin@example.com
sdk_access_key	SDK access key (SYSTEM key type)	SDK_ACCESS_KEY_123
sdk_secret_key	SDK secret key	sdk_secret_key_value

AWS Parameters (SaaS Mode)

Parameter	Description	Required	Example
account_id	AWS account ID	✓	123456789012
aws_access_key_id	AWS access key	✓	AKIAIOSFODNN7EXAMPLE
aws_secret_access_key	AWS secret key	✓	wJalrXUtnFEMI/K7MDENG/...
aws_session_token	Session token for temporary credentials	✗	FwoGZXIvYXdzE...
region	AWS region	✓	us-east-1
access_role	IAM role ARN to assume	✗	arn:aws:iam::123456789012:role/MaestroRole
mfa_serial	MFA device ARN	✗	arn:aws:iam::123456789012:mfa/user
aws_session_duration	Session duration in seconds (900-129600)	✗	3600
report_bucket	S3 bucket for reports	✓	maestro-reports

MongoDB Parameters (On-Premises/Custom Mode)

Parameter	Description	Required	Example
-----------	-------------	----------	---------

Parameter	Description	Required	Example
mongo_uris	MongoDB connection URIs (JSON)	✓	{"main": "mongodb://..."}
mongo_table_mapping	Table to connection mapping (JSON)	✗	{"secondary": ["Customers"]}
onprem_mongodb_uri	Legacy MongoDB URI	✓	mongodb://localhost:27017/maestro
onprem_mongodb_onprem_db_name	Legacy database name	✓	maestro

MinIO Parameters (On-Premises/Custom Mode)

Parameter	Description	Required	Example
onprem_minio_uri	MinIO server URI	✓	http://minio.example.com:9000
minio_report_bucket	MinIO bucket for reports	✓	maestro-reports

Vault Parameters (On-Premises/Custom Mode)

Parameter	Description	Required	Example
onprem_vault_uri	Vault server URI	✓	http://vault.example.com:8200

RabbitMQ Parameters

Parameter	Description	Required	Example
rabbit_connection_url	RabbitMQ connection URL	✓	amqp://user:pass@host:5672/
rabbit_request_queue	Request queue name	✓	m3_request_queue
rabbit_response_queue	Response queue name	✓	m3_response_queue
rabbit_api_queue	API queue for billing commands	✗	m3_api_queue
rabbit_audit_queue	Audit event queue	✗	m3_audit_queue
rabbit_timeout	Connection timeout in seconds	✗	60
rabbit_exchange	Exchange name	✗	m3_exchange

Optional Parameters

Parameter	Description	Default	Values
check_relevance	Check version compatibility	False	True, False
api_mode	Enable API mode	False	True, False
api_mode_handler_invocation	Trace ID in logs for API mode	False	True, False
send_audit	Send audit to M3 Server	False	True, False
stand_alone_audit	Store audit locally	False	True, False
rotate_logs	Rotate logs at midnight	False	True, False
logs_path	Custom log file path	\$HOME/.m3admin/logs/	Any valid path
policy_path	Custom policy file path	\$M3ADMIN_CONF/m3admin_policy.json	Any valid path
enable_modular_sdk_log	Enable Modular SDK logs	False	True, False
maestro_audit_user	Custom audit user	\$USER	Any username

Log Configuration

Configure logging through environment variables:

```
# Set M3 Admin Log Level
export M3ADMIN_LOG_LEVEL=INFO

# Set Modular SDK Log Level
export MODULAR_SDK_LOG_LEVEL=WARNING
```

Available log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL

Verify Installation and Configuration

```
# Check version
$ m3admin --version
m3admin, version 4.170.0

# Check mode
$ m3admin mode
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "{\"mode\": \"saas\"}",
  "Warnings": []
}

# Check policy
$ m3admin policy
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "[{\"Group\": \"*\", \"Resources\": \"*\", \"Type\": null, \"Effect\": \"Allow\"}]",
  "Warnings": []
}

# Test connection (if configured)
$ m3admin environment list_customers
```

[Content ↑](#)

3. Permissions and Access

Permission Model

M3 Admin implements a fine-grained, policy-based permission model that controls command execution at the group and resource level.

Policy Structure

Every M3 Admin command is verified against the policy defined in `m3admin_policy.json`. The policy consists of an array of statements, each controlling access to specific resources.

Statement Components

Each policy statement contains the following:

Component	Description	Required	Values
Effect	Allow or deny execution	✓	Allow, Deny
Group	Command group name	✓	Group name or *
Resources	List of commands	✓	Command names or *

Key Rules:

- Deny statements have higher priority than Allow statements
- * wildcard matches all groups or resources
- Wildcard patterns like `describe_*` are NOT supported - each command must be explicitly listed
- Multiple statements are evaluated in order
- If no Allow statement matches, access is denied by default

Policy Examples

Example 1: Full Access

Grant full access to all commands:

```
[
  {
    "Effect": "Allow",
    "Group": "*",
    "Resources": ["*"]
  }
]
```

Example 2: Read-Only Access

Grant read-only access (describe/list commands only):

```
[
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
      "describe",
      "describe_contacts",
      "describe_instance_quota",
      "describe_terraform",
      "describe_volume_quota",
    ]
  }
]
```



```

    "check_status"
  ]
},
{
  "Effect": "Allow",
  "Group": "environment",
  "Resources": [
    "list_customers",
    "list_regions",
    "describe_customer",
    "describe_region"
  ]
},
{
  "Effect": "Allow",
  "Group": "application",
  "Resources": [
    "describe"
  ]
},
{
  "Effect": "Deny",
  "Group": "*",
  "Resources": [
    "activate",
    "deactivate",
    "delete",
    "add",
    "update",
    "create",
    "deploy",
    "register"
  ]
}
]

```

Example 3: AWS-Only Administrator

Grant full access to AWS commands only:

```

[
  {
    "Effect": "Allow",
    "Group": "aws",
    "Resources": ["*"]
  },
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
      "describe",
      "describe_contacts",
      "describe_instance_quota",
      "describe_volume_quota",
      "check_status"
    ]
  }
],
{

```

```

    "Effect": "Allow",
    "Group": "environment",
    "Resources": [
        "list_customers",
        "list_regions",
        "describe_customer"
    ]
  },
  {
    "Effect": "Deny",
    "Group": "azure",
    "Resources": ["*"]
  },
  {
    "Effect": "Deny",
    "Group": "google",
    "Resources": ["*"]
  }
]

```

Example 4: Tenant Management Only

Grant access to tenant lifecycle operations:

```

[
  {
    "Effect": "Allow",
    "Group": "aws",
    "Resources": [
        "activate",
        "deactivate",
        "activate_region"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "azure",
    "Resources": [
        "activate",
        "deactivate",
        "activate_region"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "google",
    "Resources": [
        "activate",
        "deactivate",
        "activate_region"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
        "describe",
        "describe_contacts",

```

```

        "describe_instance_quota",
        "describe_volume_quota",
        "check_status"
    ]
}
]

```

Example 5: L3 Support Profile

Grant typical L3 support permissions:

```

[
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
      "describe",
      "describe_contacts",
      "describe_instance_quota",
      "describe_terraform",
      "describe_volume_quota",
      "check_status"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "custodian",
    "Resources": [
      "describe_activation_settings"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "setting",
    "Resources": [
      "describe"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "billing",
    "Resources": [
      "describe_azure_billing_config",
      "describe_business_units",
      "describe_daily_cost_threshold",
      "describe_month",
      "describe_tenant_business_unit"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "chef",
    "Resources": [
      "describe_configuration",
      "get_client"
    ]
  },
  {

```

```

    "Effect": "Allow",
    "Group": "permissions",
    "Resources": [
        "describe_blocked_user_config",
        "describe_group",
        "describe_mapping",
        "describe_user_permissions",
        "describe_position_from_tenant",
        "describe_title_position_mapping",
        "describe_user",
        "get_blocked_user"
    ]
},
{
    "Effect": "Allow",
    "Group": "region",
    "Resources": [
        "describe_virt_profile"
    ]
},
{
    "Effect": "Allow",
    "Group": "azure",
    "Resources": [
        "describe_diagnostic_account_storage",
        "validate_vm_counters"
    ]
},
{
    "Effect": "Allow",
    "Group": "aws",
    "Resources": [
        "validate_vm_counters",
        "reset_vm_counters"
    ]
},
{
    "Effect": "Allow",
    "Group": "openstack",
    "Resources": [
        "describe_region_metrics"
    ]
},
{
    "Effect": "Allow",
    "Group": "environment",
    "Resources": [
        "audit"
    ]
},
{
    "Effect": "Deny",
    "Group": "*",
    "Resources": [
        "delete",
        "delete_tenant_permission",
        "delete_user",

```

```

    "delete_contacts",
    "delete_instance_quota",
    "delete_volume_quota",
    "deactivate",
    "deactivate_cf_audit",
    "deactivate_ec2_audit",
    "deactivate_guarddduty",
    "deactivate_tenant_cf_audit",
    "deactivate_tenant_ec2_audit",
    "deactivate_tenant_guarddduty",
    "deactivate_region",
    "deactivate_service"
  ]
}
]

```

Example 6: Billing Administrator

Grant access to billing-related commands:

```

[
  {
    "Effect": "Allow",
    "Group": "billing",
    "Resources": [
      "add_cost_center",
      "add_custom_daily_cost_threshold",
      "add_tenant_to_customer",
      "adjustment",
      "configure",
      "configure_gcp_big_query",
      "delete_custom_daily_cost_threshold",
      "describe_azure_billing_config",
      "describe_business_units",
      "describe_daily_cost_threshold",
      "describe_month",
      "describe_tenant_business_unit",
      "export_aws",
      "export_azure",
      "export_google",
      "export_status",
      "get_currency",
      "set_azure_billing_config",
      "set_kpi",
      "update_reports_config"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "aws",
    "Resources": [
      "configure_athena",
      "manage_cost_allocation_tags"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "azure",

```

```

    "Resources": [
      "activate_service",
      "deactivate_service"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "application",
    "Resources": [
      "describe",
      "add",
      "update",
      "rotate_credentials"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "environment",
    "Resources": [
      "list_customers",
      "list_regions",
      "describe_customer"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
      "describe",
      "describe_contacts"
    ]
  }
]

```

Command Hiding and Deprecation

Command Hiding

In addition to policy-based access control, commands can be hidden from users using the `hidden` attribute in the command decorator:

```

import click
from m3_admin import check_required_param_and_convert_in_upper_case, GOOGLE_CLOUD
from m3_admin.handler.common_handler import common_handler_instance
from m3_admin.group.feature_meta.decorators import meta
from m3_admin.group.feature_meta.report_definitions import PlatformServices
from m3_admin.utils.base_click_command_interface import BaseCommand
from m3_admin.utils.decorators import ResponseDecorator, produce_audit, dry_run
from m3_admin.utils.output import CommandResponse
from modular_cli_sdk.utils.view_utils import deprecated

@deprecated(
    deprecated_date='2026-10-19',
    removal_date='2027-01-01',
    reason="Outdated logic",
)
@google.command(cls=BaseCommand, name='activate_terraform', hidden=True)
@click.option('--tenant', '-tn', nargs=1,

```

```

        callback=check_required_param_and_convert_in_upper_case,
        help='* Tenant display name')
@ResponseDecorator(click.echo, 'Can not activate terraform')
@produce_audit()
@meta(feature=PlatformServices.TERRAFORM_MANAGEMENT)
@dry_run
def activate_terraform(
    tenant: str,
) -> CommandResponse:
    """
    Creates services account keys used by Terraform Engine
    """
    return common_handler_instance().activate_terraform_from_server(
        cloud=GOOGLE_CLOUD,
        display_name=tenant,
    )

```

Effects of Hidden Commands

Aspect	Behavior
Help Output	✗ Not shown in --help output
Autocomplete	✗ Not shown in tab completion suggestions
Execution	☑ Remains executable if user knows the exact name
Policy Control	☑ Policy rules still apply and are enforced
API Access	☑ Available through Modular-API if configured

Use Cases for Hidden Commands

1. **Deprecated Commands** - Maintain backward compatibility while hiding from new users
2. **Internal/Debug Commands** - Operations for support and development teams only
3. **Beta Features** - Commands under testing not ready for general use
4. **Dangerous Operations** - Commands that should not be easily discoverable
5. **Legacy Compatibility** - Old command names preserved for existing scripts

Command Deprecation Flow

M3 Admin follows the [Modular-CLI deprecation process](#) for managing command lifecycle.

Testing Policy Configuration

Verify your policy configuration:

```

# View current policy
m3admin policy

# Try executing a command
m3admin tenant describe --tenant test-tenant

# Check available commands with current policy
m3admin --help

```

Policy Validation:

- Invalid JSON syntax will cause M3 Admin startup failure
- Invalid Effect values are ignored
- Invalid Group names are ignored
- Commands not in policy are denied by default

Policy Best Practices

1. **Principle of Least Privilege:** Grant minimum permissions needed
2. **Use Deny for Safety:** Explicitly deny dangerous operations
3. **Explicit Command Lists:** List all commands explicitly (no wildcards like `describe_*`)
4. **Document Policies:** Document policies and keep the documentation up to date
5. **Test Policies:** Verify policies work as expected before deployment
6. **Audit Access:** Review audit logs regularly
7. **Separate Policies:** Use different policies for different roles
8. **Version Control:** Store policies in version control
9. **Regular Review:** Periodically review and update policies

Documentation References

- **Modular-CLI Documentation:** <https://git.epam.com/epmc-eoos/m3-modular-cli>
- **Deprecation Flow Guide:** https://git.epam.com/epmc-eoos/m3-modular-cli/-/blob/develop/docs/development/DEPRECATION_FLOW.md
- **PyPI Package:** <https://pypi.org/project/modular-cli/>

Content ↑

AWS Credentials Resolving

M3 Admin implements a flexible, multi-tier AWS credential resolution flow that supports various authentication scenarios including MFA, role assumption, and instance profiles.

Credentials Resolution Flow

M3 Admin resolves AWS credentials following this priority order:

Flow 1: MFA + Role Assumption (Highest Security)

Requirements:

- `account_id`
- `aws_access_key_id`
- `aws_secret_access_key`
- `mfa_serial`
- `access_role`

Process:

1. Obtains session token using AWS credentials + MFA code
2. Uses session token to assume the specified IAM role
3. Caches temporary credentials locally
4. Reuses cached credentials until expiration

Example configuration:

```
account_id=123456789012
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
mfa_serial=arn:aws:iam::123456789012:mfa/john_doe
access_role=arn:aws:iam::123456789012:role/MaestroAdminRole
aws_session_duration=3600
```

Cached credentials location:


```
$M3ADMIN_CONF/<aws_access_key_id>_<expiration_date_utc>.json
```

Cached credentials format:

```
{
  "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",
  "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
  "SessionToken": "FwoGZXIvYXZzEBY...",
  "Expiration": "2025-01-20T12:00:00Z"
}
```

When to use:

- Production environments
- Multi-account access
- Compliance requirements
- Enhanced security needed

Flow 2: Session Token + Role Assumption**Requirements:**

- aws_access_key_id
- aws_secret_access_key
- aws_session_token
- access_role

Process:

1. Uses provided session token
2. Assumes specified IAM role
3. Uses role credentials for API calls

Example configuration:

```
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
aws_session_token=FwoGZXIvYXZzEBY...
access_role=arn:aws:iam::123456789012:role/MaestroAdminRole
```

When to use:

- Temporary credentials from external source
- CI/CD pipelines
- Automated workflows
- Cross-account access

Flow 3: Direct Credentials (Simple)

Requirements: * aws_access_key_id * aws_secret_access_key

Process:

1. Uses provided credentials directly
2. No role assumption
3. Direct API access

Example configuration:

```
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
region=us-east-1
```

When to use:

- Development environments
- Testing
- Simple deployments
- Single-account access

Flow 4: Instance Profile (No Configuration)**Requirements:**

- None (M3 Admin running on EC2 instance with attached IAM role)

Process:

1. Detects EC2 instance metadata service
2. Retrieves credentials from instance profile
3. Uses temporary credentials from instance role

Example configuration:

```
# Only Modular SDK credentials needed
maestro_user=admin@example.com
sdk_access_key=your_sdk_access_key
sdk_secret_key=your_sdk_secret_key

# No AWS credentials needed - uses instance profile
region=us-east-1
```

When to use:

- M3 Admin deployed on EC2
- Container deployments on ECS/EKS
- Lambda functions
- AWS-native deployments

Credentials Caching

Cache Location

Temporary credentials are cached in:

```
$M3ADMIN_CONF/<aws_access_key_id>_<expiration_timestamp>.json
```

Cache Lifecycle

- **Created:** When MFA + Role flow completes successfully
- **Reused:** For subsequent commands until expiration
- **Expired:** Automatically ignored after expiration time
- **Cleaned:** Manually delete old cache files

View Cached Credentials

```
# List cached credentials
ls -l $M3ADMIN_CONF/*.json

# View credential details
cat $M3ADMIN_CONF/ASIAIOSFODNN7EXAMPLE_20250120120000.json
```

Clear Credential Cache

```
# Remove specific cached credentials
rm $M3ADMIN_CONF/ASIAIOSFODNN7EXAMPLE_*.json

# Remove all cached credentials
rm $M3ADMIN_CONF/*.json
```

MFA Configuration

Setting Up MFA

1. **Create an MFA device** in AWS IAM Console
2. **Attach to a user** account
3. **Get ARN:** `arn:aws:iam::<account-id>:mfa/<username>`
4. **Add to configuration:**

```
mfa_serial=arn:aws:iam::123456789012:mfa/john_doe
```

Using MFA

When MFA is configured, M3 Admin prompts for MFA code:

```
$ m3admin environment list_customers
Enter MFA code: 123456
Authenticating with MFA...
Successfully authenticated
```

MFA Best Practices

- **Use hardware tokens** for production
- **Backup codes:** Store backup codes securely
- **Multiple devices:** Configure multiple MFA devices
- **Rotation:** Rotate MFA devices periodically

Role Assumption Configuration

IAM Role Setup

Create IAM role with trust relationship:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/maestro-admin"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "Bool": {
          "aws:MultiFactorAuthPresent": "true"
        }
      }
    }
  ]
}
```

Role Permissions

Attach policies to role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:*",
        "s3:*",
        "ssm:*",
        "ec2:Describe*",
        "iam:Get*",
        "iam:List*"
      ],
      "Resource": "*"
    }
  ]
}
```

Configure in M3 Admin

```
access_role=arn:aws:iam::123456789012:role/MaestroAdminRole
```

Session Duration Configuration

Control how long temporary credentials remain valid:

```
# Session duration in seconds
# Minimum: 900 (15 minutes)
# Maximum: 129600 (36 hours)
# Default: 3600 (1 hour)
aws_session_duration=7200
```

NOTE: Maximum duration depends on role configuration:

- Default roles: 1 hour maximum
- Extended duration roles: Up to 12 hours
- Configure in IAM role settings

Troubleshooting Credentials

Check Credentials Flow

```
# Enable debug logging
export M3ADMIN_LOG_LEVEL=DEBUG

# Run command and check logs
m3admin environment list_customers

# View logs
tail -f ~/.m3admin/logs/m3admin.log
```

Common Issues

Issue: MFA code rejected

```
Error: Invalid MFA code
```

Solution:

- Verify MFA device is synchronized
- Check system time is correct

- Try new MFA code
- Verify MFA serial ARN is correct

Issue: Role assumption failed

Error: User is not authorized to perform: sts:AssumeRole

Solution:

- Verify role trust relationship includes your user
- Check MFA requirement in trust policy
- Verify user has permissions to assume role

Issue: Credentials expired

Error: The security token included in the request is expired

Solution:

- Delete cached credentials: `rm $M3ADMIN_CONF/*.json`
- Re-run command to get new credentials
- Check `aws_session_duration` setting

Issue: Instance profile not found

Error: Unable to locate credentials

Solution:

- Verify EC2 instance has IAM role attached
- Check instance metadata service is accessible
- Verify IAM role has necessary permissions

[Content](#) ↑

4. Command Line Usage

Command Usage

CLI Commands Structure and Overview

The M3 Admin CLI follows a hierarchical, multi-layered command structure designed for intuitive navigation and organization.

The CLI is organized into three main levels:

- **Root Commands** - Top-level commands available directly under m3admin
- **Command Groups** - Collections of related commands (can contain both commands and subgroups)
- **Subcommands** - Specific actions within a group or subgroup

Hierarchical Organization

```

m3admin|modular_cli
├── [Root Command]      # Direct executable commands (e.g., mode, policy)
├── [Root Command]      # (e.g., enable_autocomplete)
└── [Group]             # Command groups (e.g., aws, tenant, application)
    ├── [Command]        # Commands within the group (e.g., activate)
    ├── [Command]        # (e.g., deactivate)
    └── [Subgroup]       # Nested subgroups (e.g., parent, storage)
        ├── [Command]    # Commands within subgroup (e.g., add)
        └── [Command]    # (e.g., delete)

```

Command Syntax Patterns

M3 Admin supports multiple command syntax patterns:

Pattern 1: Root Commands

```
m3admin [ROOT_COMMAND] [OPTIONS]
```

Examples:

```

m3admin --version
m3admin mode
m3admin policy
m3admin enable_autocomplete
m3admin get_operation_status --id 12345 --group aws --command activate

```

Pattern 2: Group Commands

```
m3admin [GROUP] [COMMAND] [OPTIONS]
```

Examples:

```

m3admin tenant describe --tenant prod-aws-001
m3admin aws activate --tenant prod-aws-001 --account_number 123456789012
m3admin azure deactivate --tenant prod-azure-001
m3admin environment add_customer --name "ACME Corp" --customer_id ACME

```

Pattern 3: Subgroup Commands

```
m3admin [GROUP] [SUBGROUP] [COMMAND] [OPTIONS]
```

Examples:

```
m3admin application parent add --customer_id CUSTOMER_001 --type AWS_ROLE
m3admin application parent describe --application_id APP_123
m3admin region virt_profile add --region us-east-1 --profile_name standard
```

Getting Help

You can access help documentation at any level of the command hierarchy:

```
# Show all root-level commands and groups
m3admin --help

# Show commands within a specific group
m3admin [group] --help
m3admin tenant --help

# Show commands within a subgroup
m3admin [group] [subgroup] --help
m3admin application parent --help

# Show help for a specific command
m3admin [group] [command] --help
m3admin aws activate --help
```

Help Output Example

```
$ m3admin tenant --help

Usage: m3admin tenant [OPTIONS] COMMAND [ARGS]...

    Manage tenant lifecycle operations

Options:
  --help  Show this message and exit.

Commands:
  add_contacts          Add contact emails to tenant
  check_status          Check tenant activation status
  configure_custom_chef Configure custom Chef configuration
  deactivate_region     Deactivate specific region for tenant
  delete_contacts       Delete contact emails from tenant
  delete_instance_quota Delete instance quota for tenant
  delete_volume_quota   Delete volume quota for tenant
  describe              Describe tenant configuration
  describe_contacts     Describe tenant contacts
  describe_instance_quota Describe instance quotas
  describe_volume_quota Describe volume quotas
  disable_auto_configuration Disable automatic configuration
  rename                Rename tenant
  set_instance_quota     Set instance quota for tenant
  set_volume_quota       Set volume quota for tenant
```

Navigation Tips

Tip 1: Use Tab Completion

Enable autocomplete for easier navigation:

```
# Enable autocomplete
m3admin enable_autocomplete
```

```
# Use tab to explore
m3admin [TAB]           # Shows all groups and root commands
m3admin tenant [TAB]    # Shows all tenant commands
m3admin application par[TAB] # Completes to 'parent'
```

Tip 2: Explore Hierarchically

Start broad and narrow down:

```
# 1. See all groups
m3admin --help

# 2. Explore a group
m3admin tenant --help

# 3. Get command details
m3admin tenant describe --help
```

Tip 3: Follow Naming Patterns

Commands follow consistent naming conventions:

Pattern	Purpose	Examples
describe	Get information about single resource	tenant describe, application describe
describe	Get information about specific aspect	describe_contacts, describe_quota
list	Get list of resources	list_customers, list_regions
add	Create or add resource	add, add_contacts, add_image
update	Modify existing resource	update, update_contacts
delete	Remove resource	delete, delete_contacts
activate	Enable service or resource	activate, activate_region
deactivate	Disable service or resource	deactivate, deactivate_region
set	Configure specific setting	set_quota, set_chef_mode
register	Register external resource	register_s3_bucket
configure	Configure component	configure_athena
validate	Validate configuration	validate_vm_counters
check	Check status or condition	check_status

Tip 4: Common Command Groups**Infrastructure Management:**

- aws - AWS cloud resources
- azure - Azure cloud resources
- google - Google Cloud resources
- openstack - OpenStack resources
- vsphere - VMware vSphere
- nutanix - Nutanix platform

Tenant & Access Management:

- tenant - Tenant lifecycle operations
- application - Cloud credentials management

- permissions - User and group permissions
- approval - Approval workflows

Configuration & Operations:

- environment - Environment-wide settings
- region - Region configuration
- configure - Security and audit setup
- security - Security platform integration

Monitoring & Reporting:

- billing - Cost and billing operations
- paas - PaaS services management
- workspace - Workspace management

Root Commands

Root commands provide system-level operations:

Command	Description
m3admin --version	Display M3 Admin version
m3admin mode	Display current operating mode (saas/onprem/custom)
m3admin policy	Display current permission policy
m3admin enable_autocomplete	Enable command autocompletion
m3admin disable_autocomplete	Disable command autocompletion
m3admin get_operation_status	Check long-running operation status

Root Command Examples

```
# Check version
$ m3admin --version
M3 Admin version: 4.170.0

# Check mode
$ m3admin mode
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "{\"mode\": \"saas\"}",
  "Warnings": []
}

# View policy
$ m3admin policy
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "[{\"Group\": \"*\", \"Resources\": \"*\", \"Effect\": \"Allow\"}]",
  "Warnings": []
}

# Check operation status
$ m3admin get_operation_status --id 12345-67890-abcdef --group aws --command activate
```

```
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "Operation completed successfully",
  "Warnings": []
}
```

Command Execution Examples

AWS Tenant Activation

```
m3admin aws activate \
  --tenant prod-aws-001 \
  --tenant_group production \
  --display_name "Production AWS Account" \
  --tenant_customer CUSTOMER_001 \
  --account_number 123456789012 \
  --activation_date 2025-01-20
```

Azure Tenant Activation

```
m3admin azure activate \
  --tenant prod-azure-001 \
  --tenant_group production \
  --display_name "Production Azure Subscription" \
  --tenant_customer CUSTOMER_001 \
  --subscription_id "12345678-1234-1234-1234-123456789012" \
  --tenant_id "87654321-4321-4321-4321-210987654321"
```

Add Application Credentials

```
# Add AWS role-based application
m3admin application add \
  --customer_id CUSTOMER_001 \
  --type AWS_ROLE \
  --description "Production AWS Management Role" \
  --account_number 123456789012 \
  --role MaestroManagementRole \
  --secret_key_path ./aws_credentials.json

# Add Azure credentials application
m3admin application add \
  --customer_id CUSTOMER_001 \
  --type AZURE_CREDENTIALS \
  --description "Production Azure Credentials" \
  --tenant_id "87654321-4321-4321-4321-210987654321" \
  --client_id "11111111-2222-3333-4444-555555555555" \
  --secret_key_path ./azure_credentials.json
```

Tenant Management

```
# Describe tenant
m3admin tenant describe --tenant prod-aws-001

# Set instance quota
m3admin tenant set_instance_quota \
  --tenant prod-aws-001 \
  --region us-east-1 \
  --instance_type t3.large \
  --quota 100
```

```
# Set volume quota
m3admin tenant set_volume_quota \
  --tenant prod-aws-001 \
  --region us-east-1 \
  --quota 1000

# Add contacts
m3admin tenant add_contacts \
  --tenant prod-aws-001 \
  --contacts "admin@example.com,ops@example.com"

# Check tenant status
m3admin tenant check_status --tenant prod-aws-001
```

Environment Configuration

```
# Add customer
m3admin environment add_customer \
  --name "ACME Corporation" \
  --customer_id ACME_CORP \
  --display_name "ACME Corp"

# Add region
m3admin environment add_region \
  --region_name eu-central-1 \
  --cloud AWS \
  --native_name eu-central-1

# Register S3 bucket
m3admin environment register_s3_bucket \
  --bucket_name maestro-terraform-states \
  --region us-east-1

# Register SNS topic
m3admin environment register_sns_topic \
  --topic_name maestro-notifications \
  --region us-east-1
```

Common Parameters

Most commands support these common parameters:

Parameter	Short	Description	Example
--json		JSON output format	--json
--table		Table output format	--table
--help		Display command help	--help

Long-Running Operations

Some operations run asynchronously and return an operation ID:

```
# Check operation status
$ m3admin get_operation_status \
  --id 12345-67890-abcdef-ghijk \
  --group aws \
  --command activate
```

```
# Output:
{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "Operation completed successfully",
  "Warnings": []
}
```

Command Discovery Workflow

For New Users

```
# 1. Start with help
m3admin --help

# 2. Identify relevant group
m3admin tenant --help

# 3. Find specific command
m3admin tenant describe --help

# 4. Execute command
m3admin tenant describe --tenant my-tenant
```

For Experienced Users

```
# Use autocomplete for speed
m3admin ten[TAB] des[TAB] --tenant my-[TAB]

# Or execute directly
m3admin tenant describe --tenant my-tenant --json
```

Best Practices

1. **Always check help first** - Use --help at any level to understand options
2. **Use autocomplete** - Enable autocomplete for faster command discovery
3. **Follow naming patterns** - Commands are consistently named across groups
4. **Start with describe** - Use describe commands to understand current state
5. **Use --json for scripts** - JSON output is more reliable for automation
6. **Check operation status** - Use get_operation_status for long-running operations
7. **Test in non-production** - Always test commands in development environment first
8. **Use --force carefully** - Force flag skips confirmations, use with caution

[Content ↑](#)

Output Formats

M3 Admin supports different output formats to accommodate various use cases, from interactive terminal usage to programmatic consumption. Let's explore JSON and TABLE formats in detail as the main output formats:

JSON Format

Structured JSON output suitable for programmatic consumption and API integration.

When to use:

- Automation scripts
- CI/CD pipelines
- API integration
- Log aggregation

- Data processing

Characteristics:

- Machine-readable
- Structured format
- Easy to parse
- Consistent schema

Enable: Add `--json` flag to any command

Example:

```
$ m3admin tenant describe -cid "EPAM Systems" -acc 958173500045 -c AWS
```

```
{
  "Status": "SUCCESS",
  "Code": 200,
  "table_title": "List of tenants",
  "items": [
    {
      "Tenant name": "AWS-MSTR-DEV2",
      "Cloud": "AWS",
      "Display name": "MSTR-DEV2",
      "Customer": "EPAM Systems",
      "Linked to": "",
      "Contacts": {
        "default_owner": "user_1@epam.com",
        "primary_contacts": [
          "user_2@gmail.com",
          "user_3@epam.com"
        ],
        "secondary_contacts": [],
        "tenant_manager_contacts": []
      },
      "Active": true,
      "Billing settings": {
        "deactivation_request_date": null,
        "default_for_common_cost": "AWS-EUCENTRAL",
        "last_rate_cards": null,
        "most_recent_record_date": null,
        "personal": false,
        "tenant_type": "NOT_BILLABLE"
      },
      "Account ID": "958173500045",
      "Activation Date": "2021-04-16T10:51:45.615Z",
      "Deactivation Date": ""
    }
  ],
  "Warnings": []
}
```

JSON Schema:

```
{
  "Status": "SUCCESS | ERROR",
  "Code": 200,
  "table_title": "Table title",
```

```

"items": [{"key": "value"}],
"Message": "string",
"Warnings": ["<warning1>"]
}

```

Automatic JSON mode:

- Automatically enabled when `api_mode=True` in configuration
- Useful when M3 Admin is installed as Modular-API module

Table Format

Tabular output optimized for viewing structured data in terminal.

When to use:

- Viewing lists of resources
- Comparing multiple items
- Terminal-based reports
- Quick data overview

Characteristics:

- Column-based layout
- ASCII table borders
- Header row with column names
- Fixed-width columns

Enable: Add `--table` flag to any command

Example:

```
$ m3admin environment list_customers --table
```

Existing customers

```

+-----+
|              Result              |
+-----+
| 08dfd4c6-7b14-38a8-99c9-63bed4014aab |
|-----|
|              SYSTEM              |
|-----|
| 14aa676a-3430-4018-b684-df554603a14f |
|-----|
|           Google Customer 2           |
|-----|
| 24e13914-5acc-4a21-903f-6b113bf16514 |
|-----|
|           EPAM Systems           |
+-----+

```

Output Format Comparison

Feature	CLI	JSON	Table
Human-readable	✓	✗	✓
Machine-parseable	✗	✓	~
Structured data	✗	✓	~

Feature	CLI	JSON	Table
Multiple items	~	✓	✓
Automation-friendly	✗	✓	✗
Terminal-optimized	✓	✗	✓
API integration	✗	✓	✗

Success Response Examples

CLI Success

```
$ m3admin tenant describe -cid "EPAM Systems" -acc 958173500045 -c AWS
```

```
{
  "Status": "SUCCESS",
  "Code": 200,
  "table_title": "List of tenants",
  "items": [
    {
      "Tenant name": "AWS-MSTR-DEV2",
      "Cloud": "AWS",
      "Display name": "MSTR-DEV2",
      "Customer": "EPAM Systems",
      "Linked to": "",
      "Contacts": {
        "default_owner": "user_1@epam.com",
        "primary_contacts": [
          "user_2@gmail.com",
          "user_3@epam.com"
        ],
        "secondary_contacts": [],
        "tenant_manager_contacts": []
      },
      "Active": true,
      "Billing settings": {
        "deactivation_request_date": null,
        "default_for_common_cost": "AWS-EUCENTRAL",
        "last_rate_cards": null,
        "most_recent_record_date": null,
        "personal": false,
        "tenant_type": "NOT_BILLABLE"
      },
      "Account ID": "958173500045",
      "Activation Date": "2021-04-16T10:51:45.615Z",
      "Deactivation Date": ""
    }
  ],
  "Warnings": []
}
```

Table Success

```
$ m3admin tenant describe -cid "EPAM Systems" -acc 958173500045 -c AWS --table
```

```
List of tenants
```

```
+-----+-----+-----+-----+-----+-----+
| Tenant name | Cloud | Display name | Customer | Active | Account ID |
+-----+-----+-----+-----+-----+-----+
```

AWS-MSTR-DEV2	AWS	MSTR-DEV2	EPAM Systems	True	958173500045
---------------	-----	-----------	--------------	------	--------------

Error Response Examples

CLI Error (JSON by default)

```
$ m3admin tenant describe -cid "EPAM Systems" -acc 958173500045_incorrect -c AWS

{
  "Status": "SUCCESS",
  "Code": 200,
  "Message": "Tenants not found by customer: 'EPAM Systems', cloud: 'AWS', account: '958173500045_incorrect'. Please note that the '--account' ('-acc') parameter is case sensitive. Verify the value and try again",
  "Warnings": []
}
```

Table Error

Status	Code	Response
SUCCESS	200	Tenants not found by customer: 'EPAM Systems', cloud: 'AWS', account: '958173500045_incorrect'. Please note that the '--account' ('-acc') parameter is case sensitive. Verify the value and try again

Choosing the Right Format

Use JSON format when:

- Writing automation scripts
- Integrating with other systems
- Parsing output programmatically
- Logging for later analysis
- Using in CI/CD pipelines

Use Table format when:

- Viewing lists of items
- Comparing multiple resources
- Need structured but human-readable output
- Terminal-based reporting

Output Redirection

Redirect to File

```
# JSON output
m3admin tenant describe --tenant prod-001 --json > tenant_info.json
```

```
# Table output
m3admin environment list_customers --table > customers.txt
```


Pipe to Other Commands

```
# Extract specific field using jq
m3admin tenant describe --tenant prod-001 --json | jq '.Message.account_id'

# Count rows in table
m3admin environment list_customers --table | grep -c "ACTIVE"

# Search for specific tenant
m3admin environment list_tenants --json | jq '.Message[] | select(.cloud=="AWS")'
```

Best Practices

1. **Automation:** Always use `--json` for scripts and automation
2. **Logging:** Use `--json` for log aggregation and analysis
3. **Interactive:** Use default CLI format for manual operations
4. **Reports:** Use `--table` for terminal-based reports
5. **Error Handling:** Check Status field in JSON output for error handling
6. **Parsing:** Never parse CLI or Table output programmatically
7. **Consistency:** Use the same format throughout a script or pipeline

[Content ↑](#)

Autocompletion

M3 Admin provides intelligent command autocompletion for faster command entry and discovery. Autocompletion is supported on Unix-based platforms with Bash and Zsh interpreters.

Prerequisites

- **Unix-based OS:** Linux or macOS
- **Shell:** Bash or Zsh
- **Environment Variable:** `$M3_ADMIN_HOME` must be set before installation
- **M3 Admin Version:** 4.170.0 or higher

Enable Autocompletion

1. Verify Environment

```
# Check M3_ADMIN_HOME is set
echo $M3_ADMIN_HOME
# Should output: /path/to/m3admin

# If not set, add to your shell RC file
export M3_ADMIN_HOME=/path/to/m3admin
```

2. Enable Autocompletion

```
# Enable autocompletion
m3admin enable_autocomplete
```

3. Restart Shell

```
# For Bash
source ~/.bashrc

# For Zsh
source ~/.zshrc

# Or simply restart terminal
```

Using Autocompletion

Basic Usage

Press TAB after m3admin to see all groups and root commands
 m3admin [TAB]

Suggestions:

--version	mode	tenant
--help	policy	aws
enable_autocomplete	disable_autocomplete	azure
get_operation_status	application	google
environment	region	...

Group Commands

Press TAB after group name to see commands
 m3admin tenant [TAB]

Suggestions:

describe	add_contacts	set_instance_quota
describe_contacts	check_status	set_volume_quota
delete_contacts	rename	deactivate_region
...		

Partial Matching

Type partial command and press TAB
 m3admin ten[TAB]
Completes to: m3admin tenant

m3admin tenant des[TAB]
Suggestions:
 describe describe_contacts describe_instance_quota describe_volume_quota

m3admin tenant describe_c[TAB]
Completes to: m3admin tenant describe_contacts

Subgroup Commands

Works with subgroups too
 m3admin application [TAB]

Suggestions:

add	delete	describe	parent	rotate_credentials	update
-----	--------	----------	--------	--------------------	--------

m3admin application parent [TAB]

Suggestions:

add	delete	describe	update
-----	--------	----------	--------

Autocompletion Examples

Example 1: Discover AWS Commands

\$ m3admin aws [TAB]

activate	deactivate	configure_athena
activate_region	add_image	manage_cost_allocation_tags
validate_vm_counters	reset_vm_counters	...

Example 2: Fast Command Entry

```
# Instead of typing the full command
m3admin tenant describe --tenant prod-aws-001

# Use TAB completion
m3admin ten[TAB] des[TAB] --tenant prod-aws-001
#           ↑ completes to tenant
#           ↑ completes to describe
```

Example 3: Explore Unknown Groups

```
# Not sure what commands are in 'environment'?
$ m3admin environment [TAB]

add_customer      list_customers    describe_customer
add_region        list_regions      describe_region
register_s3_bucket register_sns_topic audit
...
```

Autocompletion Modes

M3 Admin detects installation mode and configures autocompletion accordingly:

Mode 1: System-Wide (M3 Admin Instance)

When: M3 Admin installed on dedicated instance/server

Configuration:

- Script installed to: /etc/profile.d/
- Script name: profile_bash_m3admin_complete.sh or profile_zsh_m3admin_complete.sh
- Sourced automatically for all users on new terminal sessions
- No per-user configuration needed

Verification:

```
# Check if script exists
ls -la /etc/profile.d/*m3admin*

# Expected:
-rw-r--r-- 1 root root 2048 Jan 20 10:30 /etc/profile.d/profile_bash_m3admin_complete.sh
```

Mode 2: User-Specific (Local Installation)

When: M3 Admin installed locally by user

Configuration:

- Script installed to: \$M3_ADMIN_HOME/
- Script name: bash_m3admin_complete.sh or zsh_m3admin_complete.sh
- RC file (~/.bashrc or ~/.zshrc) modified to source script
- Only available for installing user

Verification:

```
# Check if script exists
ls -la $M3_ADMIN_HOME/*complete*

# Check RC file modification
```

```
tail ~/.bashrc # for Bash
# or
tail ~/.zshrc  # for Zsh

# Should see:
# M3 Admin autocompletion
source $M3_ADMIN_HOME/bash_m3admin_complete.sh
```

Disable Autocompletion

```
# Disable autocompletion
m3admin disable_autocomplete
```

What gets removed:

- System-wide mode: Script removed from /etc/profile.d/
- User-specific mode: Script removed from \$M3_ADMIN_HOME/ and RC file cleaned

Autocompletion Features

Feature	Bash	Zsh	Description
Root commands	✓	✓	Complete root-level commands
Groups	✓	✓	Complete command groups
Group commands	✓	✓	Complete commands within groups
Subgroups	✓	✓	Complete subgroups and their commands
Partial matching	✓	✓	Complete from partial input
Case sensitivity	✓	✓	Case-sensitive completion
Descriptions	✗	✓	Show command descriptions (Zsh only)

Bash vs Zsh Autocompletion

Bash Autocompletion

```
$ m3admin tenant [TAB]
add_contacts      describe          set_instance_quota
check_status      describe_contacts set_volume_quota
delete_contacts   rename           deactivate_region
```

Characteristics:

- Simple list of completions
- Alphabetically sorted
- No descriptions

Zsh Autocompletion

```
$ m3admin tenant [TAB]

-- m3admin tenant commands --
add_contacts      -- Add contact emails to tenant
check_status      -- Check tenant activation status
delete_contacts   -- Delete contact emails from tenant
describe          -- Describe tenant configuration
describe_contacts -- Describe tenant contacts
rename            -- Rename tenant
set_instance_quota -- Set instance quota for tenant
set_volume_quota  -- Set volume quota for tenant
```

Characteristics:

- Descriptions for each command
- Grouped by category
- Enhanced visual formatting
- More context for command selection

Troubleshooting Autocompletion**Issue: Autocompletion is not working after enabling****Solution:**

```
# Restart shell
exec $SHELL

# Or source RC file
source ~/.bashrc # Bash
source ~/.zshrc  # Zsh

# Or restart terminal
```

Issue: “M3_ADMIN_HOME not set” error**Solution:**

```
# Set M3_ADMIN_HOME before enabling
export M3_ADMIN_HOME=/path/to/m3admin

# Add to RC file permanently
echo 'export M3_ADMIN_HOME=/path/to/m3admin' >> ~/.bashrc
source ~/.bashrc
```

Issue: Completions show incorrect commands**Solution:**

```
# Disable and re-enable autocompletion
m3admin disable_autocomplete
m3admin enable_autocomplete

# Restart shell
exec $SHELL
```

Issue: Permission denied when enabling (system-wide mode)**Solution:**

```
# System-wide installation requires sudo
sudo m3admin enable_autocomplete

# Or use user-specific mode (no sudo needed)
# Installation automatically detects and uses user mode if no sudo access
```

Issue: Autocompletion is slow or laggy**Possible causes:**

- Large command set
- Slow disk I/O
- Network-mounted home directory

Solution:

```
# Disable autocompletion if performance is poor
m3admin disable_autocomplete

# Consider upgrading to faster storage
# Or use alias shortcuts instead
```

Best Practices

1. **Enable Early:** Enable autocompletion during initial setup
2. **Use TAB Generously:** Press TAB frequently to discover commands
3. **Learn Patterns:** Understand command naming patterns for faster completion
4. **Combine with `--help`:** Use [TAB] `--help` to learn about commands
5. **Keep Updated:** Re-enable after M3 Admin upgrades to get new commands

Autocompletion Limitations

- **Unix-only:** Not available on Windows
- **Bash 4.0+:** Older Bash versions not supported
- **Zsh 5.0+:** Older Zsh versions not supported
- **No dynamic completion:** Doesn't complete tenant names, region names, etc.
- **No option completion:** Doesn't complete command options (`--tenant`, `--region`, etc.)

Alternative: Manual Completion Setup

If automatic setup fails, manually configure autocompletion:

Bash Manual Setup

```
# Create completion script
m3admin --completion bash > ~/.m3admin_completion.bash

# Add to ~/.bashrc
echo 'source ~/.m3admin_completion.bash' >> ~/.bashrc

# Reload
source ~/.bashrc
```

Zsh Manual Setup

```
# Create completion script
m3admin --completion zsh > ~/.m3admin_completion.zsh

# Add to ~/.zshrc
echo 'source ~/.m3admin_completion.zsh' >> ~/.zshrc

# Reload
source ~/.zshrc
```

[Content ↑](#)

5. Cloud Provider Management

M3 Admin provides comprehensive multi-cloud management capabilities across major cloud providers. This section covers tenant activation, configuration, and management for each supported cloud platform.

Supported Cloud Providers

Provider	Group Command	Primary Use Cases
Amazon Web Services (AWS)	aws	Public cloud, compute, storage, managed services
Microsoft Azure	azure	Public cloud, enterprise integration
Google Cloud Platform (GCP)	google	Public cloud, data analytics, ML
OpenStack	openstack	Private cloud, on-premises
VMware vSphere	vsphere	Private cloud, virtualization
VMware vCloud Director	vclouddirector	Private cloud, multi-tenancy
Nutanix	nutanix	Hyper-converged infrastructure
Hardware Cloud	hardware	Bare-metal provisioning
Enterprise Cloud	enterprise	Enterprise-specific clouds

AWS Cloud Management

Activate an AWS Tenant

```
m3admin aws activate \
  --tenant prod-aws-001 \
  --tenant_group production \
  --display_name "Production AWS Account" \
  --tenant_customer CUSTOMER_001 \
  --account_number 123456789012 \
  --activation_date 2025-01-20
```

Optional parameters:

- `--read_only` - Mark tenant as read-only
- `--personal` - Mark as personal account
- `--project_type` - Set project type (billable, not_billable, not_defined)
- `--paying_account_number` - Specify paying account for consolidated billing
- `--unreachable` - Mark tenant as unreachable

AWS-Specific Operations

```
# Configure AWS Athena for billing
m3admin aws configure_athena \
  --tenant prod-aws-001 \
  --database_name maestro_billing \
  --output_location s3://maestro-athena-results/

# Manage cost allocation tags
m3admin aws manage_cost_allocation_tags \
  --tenant prod-aws-001 \
  --tags "Environment,Project,CostCenter"

# Add custom AMI
m3admin aws add_image \
```

```
--tenant prod-aws-001 \  
--region us-east-1 \  
--image_id ami-0abcdef1234567890 \  
--image_name "Custom Ubuntu 20.04"
```

Validate VM counters

```
m3admin aws validate_vm_counters --tenant prod-aws-001
```

Reset VM counters

```
m3admin aws reset_vm_counters --tenant prod-aws-001
```

Deactivate an AWS Tenant

```
m3admin aws deactivate --tenant prod-aws-001
```

OpenStack Management

Activate an OpenStack Tenant

```
m3admin openstack activate \  
--tenant prod-openstack-001 \  
--tenant_group production \  
--display_name "Production OpenStack Project" \  
--tenant_customer CUSTOMER_001 \  
--project_id openstack_project_id \  
--activation_date 2025-01-20
```

OpenStack-Specific Operations

Activate region

```
m3admin openstack activate_region \  
--tenant prod-openstack-001 \  
--region RegionOne
```

Add image

```
m3admin openstack add_image \  
--tenant prod-openstack-001 \  
--image_id image-uuid-here \  
--image_name "Ubuntu 20.04"
```

Add public image

```
m3admin openstack add_public_image \  
--image_id image-uuid-here \  
--image_name "Ubuntu 20.04 Public"
```

Add region metrics configuration

```
m3admin openstack add_region_metrics \  
--tenant prod-openstack-001 \  
--region RegionOne \  
--metrics_endpoint https://metrics.openstack.local
```

Describe region metrics

```
m3admin openstack describe_region_metrics \  
--tenant prod-openstack-001 \  
--region RegionOne
```

Reset VM counters

```
m3admin openstack reset_vm_counters --tenant prod-openstack-001
```


Deactivate an OpenStack Tenant

```
m3admin openstack deactivate --tenant prod-openstack-001
```

Cross-Cloud Operations

Some operations work across all cloud providers:

Tenant Information

```
# Describe tenant (works for all clouds)
m3admin tenant describe --tenant <tenant-name>

# Check tenant status
m3admin tenant check_status --tenant <tenant-name>

# List all tenants
m3admin environment list_tenants
```

Quota Management

```
# Set instance quota (AWS, Azure, GCP, OpenStack)
m3admin tenant set_instance_quota \
  --tenant <tenant-name> \
  --region <region> \
  --instance_type <type> \
  --quota 100

# Set volume quota (AWS, Azure, GCP, OpenStack)
m3admin tenant set_volume_quota \
  --tenant <tenant-name> \
  --region <region> \
  --quota 10000

# Describe quotas
m3admin tenant describe_instance_quota --tenant <tenant-name>
m3admin tenant describe_volume_quota --tenant <tenant-name>
```

Contact Management

```
# Add contacts (all clouds)
m3admin tenant add_contacts \
  --tenant <tenant-name> \
  --contacts "admin@example.com,ops@example.com"

# Describe contacts
m3admin tenant describe_contacts --tenant <tenant-name>

# Delete contacts
m3admin tenant delete_contacts \
  --tenant <tenant-name> \
  --contacts "ops@example.com"
```

Application Credentials Management

Add Cloud Credentials

```
# AWS Role-based credentials
m3admin application add \
  --customer_id CUSTOMER_001 \
```

```
--type AWS_ROLE \  
--description "AWS Management Role" \  
--account_number 123456789012 \  
--role MaestroManagementRole \  
--secret_key_path ./aws_role.json  
  
# AWS Access Keys  
m3admin application add \  
  --customer_id CUSTOMER_001 \  
  --type AWS_KEYS \  
  --description "AWS Access Keys" \  
  --account_number 123456789012 \  
  --secret_key_path ./aws_keys.json  
  
# Azure credentials  
m3admin application add \  
  --customer_id CUSTOMER_001 \  
  --type AZURE_CREDENTIALS \  
  --description "Azure Service Principal" \  
  --tenant_id "tenant-uuid" \  
  --client_id "client-uuid" \  
  --secret_key_path ./azure_creds.json  
  
# GCP credentials  
m3admin application add \  
  --customer_id CUSTOMER_001 \  
  --type GOOGLE \  
  --description "GCP Service Account" \  
  --project_id my-gcp-project \  
  --secret_key_path ./gcp_service_account.json
```

Manage Credentials

```
# Describe application  
m3admin application describe --application_id APP_ID  
  
# Rotate credentials  
m3admin application rotate_credentials --application_id APP_ID  
  
# Update application  
m3admin application update \  
  --application_id APP_ID \  
  --description "Updated description"  
  
# Delete application  
m3admin application delete --application_id APP_ID
```

Best Practices

Activation

1. **Plan Activation Date:** Use `--activation_date` for billing alignment
2. **Use Display Names:** Always provide meaningful `--display_name`
3. **Organize Tenant Groups:** Use `--tenant_group` for logical organization
4. **Document Tenants:** Add contacts to tenant settings immediately after activation
5. **Set Quotas Early:** Configure quotas before production use

Security

1. **Use Role-Based Access:** Prefer IAM roles over access keys (AWS)
2. **Rotate Credentials:** Regularly rotate application credentials
3. **Limit Permissions:** Grant minimum required permissions
4. **Monitor Access:** Review audit logs regularly
5. **Read-Only Tenants:** Use `--read_only` flag for compliance tenants

Operations

1. **Check Status First:** Always verify tenant status before performing any operations
2. **Use JSON for Automation:** Use `--json` flag in scripts
3. **Monitor Long Operations:** Check `get_operation_status` for async operations
4. **Test in Non-Prod:** Test commands in development tenants first
5. **Backup Configurations:** Export tenant configurations before major changes

[Content](#) ↑

6. Audit Service

M3 Admin provides comprehensive logging capabilities to enable audit and tracking for all administrative operations, ensuring compliance, security, and operational transparency.

Audit Overview

All M3 Admin command executions are recorded and can be stored in multiple ways:

- **Standalone Audit:** Store audit events locally in DynamoDB (SaaS mode) or MongoDB (On-Premises mode)
- **Server Audit:** Send audit events to M3 Server via RabbitMQ
- **Both:** Enable both standalone and server audit simultaneously

Audit Configuration

Configure audit behavior in `m3admin.conf`:

```
# Store audit events locally in database
stand_alone_audit=True

# Send audit events to M3 Server via RabbitMQ
send_audit=True

# RabbitMQ audit queue (required if send_audit=True)
rabbit_audit_queue=m3_audit_queue

# Custom audit user (useful in containerized environments)
maestro_audit_user=admin
```

Audit Table Structure

Table Name

- **SaaS Mode:** DynamoDB table named `M3adminAudit`
- **On-Premises Mode:** MongoDB collection named `M3adminAudit`

Primary Keys

Key Type	Attribute	Type	Description
Partition Key	user	String	User who executed the command
Sort Key	timestamp	String	ISO 8601 timestamp of execution

Global Secondary Indexes

Index Name	Partition Key	Sort Key	Purpose
group-index	group (String)	timestamp (String)	Query by command group
command-index	command (String)	timestamp (String)	Query by command name

Audit Record Attributes

Attribute	Type	Description	Example
user	String	User who executed command	admin@example.com
timestamp	String	Execution timestamp (ISO 8601)	2025-01-20T10:30:45Z
group	String	Command group	aws, tenant, application

Attribute	Type	Description	Example
command	String	Command name	activate, describe, add
parameters	Map	Command parameters (secured)	{"tenant": "prod-001"}
result	String	Command execution result	Success message or error
status	String	Execution status	SUCCESS, ERROR
execution_time	Number	Execution time in milliseconds	1250
ip_address	String	Client IP address	192.168.1.100
session_id	String	Session identifier	session-uuid

Note: Sensitive parameters (passwords, keys) are automatically secured and not stored in audit records.

Querying Audit Events

Use the `m3admin environment audit` command to query audit events:

Basic Usage

```
# View all audit events (last 24 hours)
m3admin environment audit

# View audit events for specific user
m3admin environment audit --user admin@example.com

# View audit events for specific command group
m3admin environment audit --group aws

# View audit events for specific command
m3admin environment audit --command activate
```

Time-Based Filtering

```
# Events from specific date
m3admin environment audit --from_date 2025-01-01

# Events within date range
m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31

# Events from last 7 days
m3admin environment audit --from_date $(date -d '7 days ago' +%Y-%m-%d)
```

Combined Filters

```
# AWS commands by specific user in date range
m3admin environment audit \
  --user admin@example.com \
  --group aws \
  --from_date 2025-01-01 \
  --to_date 2025-01-31

# Failed commands for specific group
m3admin environment audit \
  --group tenant
```

Output Formats

```
# JSON output for programmatic processing
m3admin environment audit \
  --user admin@example.com \
  --from_date 2025-01-01 \
  --json

# Table format for terminal viewing
m3admin environment audit \
  --group aws \
  --from_date 2025-01-20 \
  --table
```

Save to File

```
# Save audit report locally
m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --save_to /path/to/audit_report.json

# Save in CSV format
m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --format csv \
  --save_to /path/to/audit_report.csv
```

Email Reports

```
# Send audit report via email
m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --email admin@example.com,security@example.com \
  --email_subject "M3 Admin Monthly Audit Report - January 2025"
```

Audit Examples

Example 1: Retrieving Monthly Audit

Generate comprehensive monthly audit report:

```
m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --limit 100 \
  --email responsible@example.com \
  --json > /reports/audit_january_2025.json
```

Example 2: User Activity Review

Review specific user's activities:

```
m3admin environment audit \
  --user john_doe \
  --table
```

Sample Output:

The command's response is pretty huge and the result table structure can be broken.
Do you want to show the response in the JSON format? [y/n]: y

List of audit events

```
{
  "Status": "SUCCESS",
  "Code": 200,
  "table_title": "List of audit events",
  "items": [
    {
      "User": "john_doe",
      "Timestamp": "25.03.2024 08:53:00",
      "Group": "billing",
      "Command": "delete_kpi",
      "Parameters": {
        "month": 3,
        "region": "EPAM-MOB3",
        "day": null,
        "year": 2024,
        "tenant": "EPM-CIT2"
      },
      "Success": false,
      "Result": "No records found",
      "Warnings": null
    },
    {
      "User": "john_doe",
      "Timestamp": "25.03.2024 09:08:27",
      "Group": "billing",
      "Command": "delete_kpi",
      "Parameters": {
        "month": 3,
        "region": "EPAM-MOB3",
        "day": null,
        "year": 2024,
        "tenant": "EPM-CIT2"
      },
      "Success": true,
      "Result": "Records successfully removed",
      "Warnings": ""
    },
    {
      "User": "john_doe",
      "Timestamp": "25.03.2024 09:08:30",
      "Group": "billing",
      "Command": "delete_kpi",
      "Parameters": {
        "month": 3,
        "region": "EPAM-MOB3",
        "day": null,
        "year": 2024,
        "tenant": "EPM-CIT2"
      },
      "Success": false,
      "Result": "No records found",
      "Warnings": null
    }
  ]
}
```

```

    "User": "john_doe",
    "Timestamp": "08.04.2024 14:39:47",
    "Group": "billing",
    "Command": "delete_kpi",
    "Parameters": {
      "month": 3,
      "region": "EPAM-MOB3",
      "day": null,
      "year": 2024,
      "tenant": "EPM-CIT2"
    },
    "Success": false,
    "Result": "502:Connection to RabbitMQ refused. Bad credentials.",
    "Warnings": null
  },
  "Warnings": []
}

```

Output Fields:

- **User:** Username who performed the operation
- **Timestamp:** When the operation was executed (format: DD.MM.YYYY HH:MM:SS)
- **Group:** Command group category (e.g., billing, tenant, user)
- **Command:** Specific command that was executed
- **Parameters:** Full parameters passed to the command
- **Success:** Boolean indicating if operation succeeded
- **Result:** Success message or error description
- **Warnings:** Any warnings generated during execution

Note: When the table output is too large, the CLI prompts you to switch to JSON format for better readability.

Example 3: Failed Operations Investigation

Find all failed operations in the last 7 days:

```

m3admin environment audit \
  --from_date $(date -d '7 days ago' +%Y-%m-%d) \
  --limit 100 \
  --json | jq '.items[] | select(.Success == false) | {Timestamp, User, Group, Command, Result}'

```

Sample Output:

```

{
  "Timestamp": "25.03.2024 08:52:30",
  "User": "john_doe",
  "Group": "billing",
  "Command": "delete_kpi",
  "Result": "502:Connection to RabbitMQ refused. Bad credentials."
}
{
  "Timestamp": "25.03.2024 08:53:00",
  "User": "john_doe",
  "Group": "billing",
  "Command": "delete_kpi",
  "Result": "No records found"
}

```



```
{
  "Timestamp": "25.03.2024 09:09:32",
  "User": "john_doe",
  "Group": "billing",
  "Command": "delete_kpi",
  "Result": "ModularUser 'meta' must contain 'service_name' and 'service_display_name'"
}
```

Description: Filters audit events to show only failed operations (`Success == false`), displaying key information for troubleshooting.

Example 4: Tenant-Specific Audit

Audit all operations for specific tenants:

```
m3admin environment audit \
  --tenant EPM-TENANT-01 \
  --tenant EPM-TENANT-02 \
  --from_date 2025-01-01 \
  --limit 50 \
  --json
```

Example 5: Command Group Monitoring

Monitor all billing-related commands:

```
m3admin environment audit \
  --group billing \
  --from_date 2025-01-01 \
  --reverse_order \
  --table
```

Secured Parameters

The following parameter types are automatically secured (not stored in audit):

- Passwords
- Secret keys
- Access keys
- API tokens
- Private keys
- Connection strings containing credentials
- Any parameter containing “password”, “secret”, “key”, “token” in name

Audit Best Practices

For Administrators

1. **Enable Both Audit Methods:** Use both standalone and server audit for redundancy
2. **Regular Reviews:** Schedule regular audit reviews (weekly/monthly)
3. **Automated Reports:** Set up automated audit report emails
4. **Retention Policy:** Define and implement audit retention policies
5. **Access Control:** Limit audit query access to authorized personnel only

For Security Teams

1. **Monitor Failed Operations:** Regular check for failed authentication or authorization
2. **Unusual Patterns:** Look for unusual command patterns or timing

3. **Privileged Operations:** Closely monitor tenant activations and credential operations
4. **Cross-Reference:** Compare audit logs with other security logs
5. **Incident Response:** Use audit logs for security incident investigations

For Compliance

1. **Comprehensive Logging:** Ensure all operations are audited
2. **Long-Term Storage:** Archive audit logs per compliance requirements
3. **Immutable Logs:** Store audit logs in tamper-proof storage
4. **Regular Reporting:** Generate compliance reports on schedule
5. **Evidence Collection:** Maintain audit logs as evidence for audits

Troubleshooting Audit

Issue: Audit events not appearing

Check:

```
# Verify audit is enabled
grep -E "stand_alone_audit|send_audit" $M3ADMIN_CONF/m3admin.conf

# Check RabbitMQ queue (if using send_audit)
# Verify rabbit_audit_queue is configured

# Check database access
# Verify DynamoDB/MongoDB permissions
```

Enable debug logging:

```
export M3ADMIN_LOG_LEVEL=DEBUG
m3admin <command>
tail -f ~/.m3admin/logs/m3admin.log | grep -i audit
```

Issue: Cannot query audit events

Check permissions:

```
# Verify policy allows environment audit command
m3admin policy | grep -A5 '"Group": "environment"'

# Should include "audit" in Resources
```

Issue: Audit table doesn't exist

SaaS Mode (DynamoDB):

```
# Check if table exists
aws dynamodb describe-table --table-name M3adminAudit --region <region>

# If not, contact system administrator to create table
```

On-Premises Mode (MongoDB):

```
# Collection is created automatically on first audit event
# Verify MongoDB connection is working
```

Audit Event Decorator

For developers extending M3 Admin, use the audit decorator:

```
@environment.command(cls=BaseCommand, name='add_application')
# ...
@click.option('--customer_id', '-cid', nargs=1,
              callback=check_required_param,
              help='* Customer ID')
# ...
@ResponseDecorator(
    click.echo, 'Can not add application', secured_params=['secret_key'],
)
@produce_audit(secured_params=['secret_key'])
@dry_run
def add_application(
    customer_id: str,
    type: str,
    meta: str,
    description: str,
    secret_key: str | None = None,
) -> CommandResponse:
    """
    Adds application entity
    """
    return environment_handler_instance().add_application_handler(
        customer_id=customer_id,
        type=type,
        meta=meta,
        description=description,
        secret_key=secret_key,
    )
```

Important: The `@produce_audit()` decorator must be placed **under** the `@ResponseDecorator()` to ensure proper audit event generation.

[Content ↑](#)

7. Advanced Configuration

Project Conventions

Naming Conventions

String Quotes:

- Use single quotes (') in command interface definitions (src/group/*.py)
- Required for Modular-API command generator parsing
- Ensures proper API command generation

Parameter Naming:

- Use underscores for parameter names: tenant_name, account_id
- Do NOT use dashes: tenant-name ✗
- Maintains consistency with Python function parameters

Code Organization

```

m3admin/
├── src/
│   ├── m3_admin/                                # Main package
│   │   ├── __init__.py
│   │   ├── aws/                                # AWS cloud provider
│   │   │   ├── __init__.py
│   │   │   ├── connection/
│   │   │   └── service/
│   │   ├── azure/                             # Azure cloud provider
│   │   │   ├── __init__.py
│   │   │   ├── connection/
│   │   │   └── service/
│   │   ├── google/                           # Google cloud provider
│   │   │   └── ...
│   │   ├── group/                             # Command groups
│   │   │   ├── __init__.py
│   │   │   ├── application.py                # Application management
│   │   │   ├── aws.py                        # AWS commands
│   │   │   ├── azure.py                     # Azure commands
│   │   │   ├── tenant.py                    # Tenant management
│   │   │   ├── feature_meta/                # Feature metadata
│   │   │   │   ├── decorators.py            # Meta decorators
│   │   │   │   └── report_definitions.py
│   │   ├── handler/                          # Business logic handlers
│   │   │   ├── aws_group_handler.py
│   │   │   ├── common_handler.py
│   │   │   └── ...
│   │   ├── utils/                            # Utilities
│   │   │   ├── decorators.py                # Response, audit, dry_run decorators
│   │   │   ├── base_click_command_interface.py
│   │   │   ├── file_utils.py
│   │   │   ├── output.py
│   │   │   ├── time_utils.py
│   │   │   └── ...
│   │   ├── commons/                         # Common modules
│   │   ├── config/                          # Configuration
│   │   └── credentials/                     # Credential management

```

├── docs/	# Documentation
├── m3_admin_complete/	# Shell completion
├── tests/	# Test suite
├── api_module.json	# API module definition
├── pyproject.toml	# Project configuration
├── requirements-dev.txt	# Development dependencies
├── tox.ini	# Tox configuration
├── CHANGELOG.md	
└── README.md	

Key Components:

- **group/** - Click command groups organized by cloud provider and functionality
- **handler/** - Business logic separated from CLI command definitions
- **utils/decorators.py** - Contains ResponseDecorator, produce_audit, dry_run, etc.
- **aws/, azure/, google/** - Cloud provider-specific connection and service modules
- **credentials/** - Credential management services
- **config/** - Configuration handling
- **commons/** - Shared utilities and common code

Decorator Flow Example (from aws.py):

```
from m3_admin.utils.decorators import ResponseDecorator, produce_audit, dry_run
from m3_admin.group.feature_meta.decorators import meta

@aws.command(cls=BaseCommand, name='reset_vm_counters')
@click.option('--tenant', '-tn', nargs=1)
@ResponseDecorator(click.echo, 'Can not reset vm counters.')
@produce_audit()
@meta(feature=PlatformServices.VM_MANAGEMENT)
@dry_run
def reset_vm_counters(tenant):
    """Command implementation"""
    return aws_handler_instance().validate_and_sync_aws_counter(...)
```

Important Rules:

1. @produce_audit() must be **under** @ResponseDecorator()
2. Specify sensitive parameters in secured_params list
3. Null/error results won't be audited if decorator order is wrong

Multiple Database Connections (MongoDB)

M3 Admin supports connecting to multiple MongoDB databases simultaneously.

Configuration

```
# Define multiple MongoDB connections
mongo_uris={"main": "mongodb+srv://user:pass@host1/db1", "secondary":
"mongodb+srv://user:pass@host2/db2", "billing": "mongodb://localhost:27017/billing_db"}

# Map tables/collections to specific connections
mongo_table_mapping={"secondary": ["Customers", "Tenants"], "billing": ["BillingRecords",
"Invoices"]}]

# Tables not in mapping use "main" connection by default
# In this example: SdkKeys, Applications, Regions use "main"
```

Use Cases

Scenario 1: Separate Customer Database

```
mongo_uris={"main": "mongodb://prod-primary/maestro", "customers": "mongodb://customer-
db/customers"}
mongo_table_mapping={"customers": ["Customers", "Tenants", "Contacts"]}
```

Scenario 2: Billing Database Separation

```
mongo_uris={"main": "mongodb://prod-primary/maestro", "billing": "mongodb://billing-
server/billing"}
mongo_table_mapping={"billing": ["BillingRecords", "Invoices", "CostCenters"]}
```

Scenario 3: Geographic Distribution

```
mongo_uris={"main": "mongodb://us-east/maestro", "eu": "mongodb://eu-central/maestro", "asia":
"mongodb://ap-southeast/maestro"}
mongo_table_mapping={"eu": ["Customers_EU", "Tenants_EU"], "asia": ["Customers_ASIA",
"Tenants_ASIA"]}
```

Default Behavior

- All tables use the "main" connection by default
- Only tables listed in `mongo_table_mapping` use alternate connections
- If `mongo_table_mapping` is not specified, all tables use "main"

Custom Mode Combinations

Custom mode allows mixing database, blob storage, and secrets management:

Example 1: MongoDB + S3 + SSM

```
mode=custom
custom_mode_db=mongodb
custom_mode_blob=s3
custom_mode_secret=ssm

# MongoDB configuration
mongo_uris={"main": "mongodb://localhost:27017/maestro"}

# AWS configuration for S3 and SSM
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
region=us-east-1
report_bucket=maestro-reports
```

Use case: On-premises database with cloud storage and secrets

Example 2: DynamoDB + MinIO + Vault

```
mode=custom
custom_mode_db=dynamodb
custom_mode_blob=minio
custom_mode_secret=vault

# AWS configuration for DynamoDB
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
region=us-east-1
```

```
# MinIO configuration
onprem_minio_uri=http://minio.internal.com:9000
minio_report_bucket=maestro-reports
```

```
# Vault configuration
onprem_vault_uri=http://vault.internal.com:8200
```

Use case: Cloud database with on-premises storage and secrets

Example 3: All On-Premises

```
mode=custom
custom_mode_db=mongodb
custom_mode_blob=minio
custom_mode_secret=vault

# MongoDB configuration
mongo_uris={"main": "mongodb://mongodb.internal.com:27017/maestro"}

# MinIO configuration
onprem_minio_uri=http://minio.internal.com:9000
minio_report_bucket=maestro-reports

# Vault configuration
onprem_vault_uri=http://vault.internal.com:8200
```

Use case: Fully air-gapped on-premises deployment

Environment-Specific Configuration Examples

Development Configuration

```
# m3admin.conf for development

maestro_user=dev@example.com
sdk_access_key=DEV_SDK_KEY
sdk_secret_key=DEV_SDK_SECRET

mode=saas
aws_access_key_id=DEV_AWS_KEY
aws_secret_access_key=DEV_AWS_SECRET
region=us-east-1
report_bucket=maestro-dev-reports

rabbit_connection_url=amqp://dev:password@rabbit-dev.internal:5672/
rabbit_request_queue=m3_dev_request
rabbit_response_queue=m3_dev_response

# Enable verbose logging
M3ADMIN_LOG_LEVEL=DEBUG
MODULAR_SDK_LOG_LEVEL=DEBUG

# Disable version check for local testing
check_relevance=False

# Enable standalone audit for testing
stand_alone_audit=True
send_audit=False
```

Production Configuration

```
# m3admin.conf for production

maestro_user=admin@example.com
sdk_access_key=PROD_SDK_KEY
sdk_secret_key=PROD_SDK_SECRET

mode=saas
account_id=123456789012
aws_access_key_id=PROD_AWS_KEY
aws_secret_access_key=PROD_AWS_SECRET
region=us-east-1
access_role=arn:aws:iam::123456789012:role/MaestroProdRole
mfa_serial=arn:aws:iam::123456789012:mfa/admin
aws_session_duration=3600

report_bucket=maestro-prod-reports

rabbit_connection_url=amqp://prod:password@rabbit-prod.internal:5672/
rabbit_request_queue=m3_prod_request
rabbit_response_queue=m3_prod_response
rabbit_audit_queue=m3_prod_audit

# Production logging
M3ADMIN_LOG_LEVEL=INFO
MODULAR_SDK_LOG_LEVEL=WARNING

# Enable version check
check_relevance=True

# Enable both audit methods
stand_alone_audit=True
send_audit=True

# Enable log rotation
rotate_logs=True
logs_path=/var/log/m3admin/
```

Security Hardening

Restrict Permissions

```
[
  {
    "Effect": "Allow",
    "Group": "tenant",
    "Resources": [
      "describe",
      "describe_contacts",
      "describe_instance_quota",
      "describe_volume_quota",
      "check_status"
    ]
  },
  {
    "Effect": "Allow",
    "Group": "environment",
```



```

    "Resources": [
      "list_customers",
      "list_tenants",
      "audit"
    ]
  },
  {
    "Effect": "Deny",
    "Group": "*",
    "Resources": [
      "delete",
      "delete_contacts",
      "delete_instance_quota",
      "delete_volume_quota",
      "delete_tenant_permission",
      "delete_user",
      "deactivate",
      "deactivate_region",
      "deactivate_service",
      "deactivate_cf_audit",
      "deactivate_ec2_audit",
      "deactivate_guarddduty",
      "deactivate_tenant_cf_audit",
      "deactivate_tenant_ec2_audit",
      "deactivate_tenant_guarddduty",
      "remove",
      "remove_contacts",
      "remove_quota"
    ]
  }
]

```

Note: Wildcard patterns (e.g., `delete_*`) are not supported in M3 Admin policies. Each command must be explicitly listed. Adjust the Resources list based on your specific security requirements and available commands.

Credential Rotation

```

# Rotate application credentials
m3admin application rotate_credentials --application_id APP_ID

```

```

# Note: SDK credentials are managed through Modular-SDK
# Contact your administrator to rotate SDK credentials

```

```

# Rotate AWS credentials (manual process)
# 1. Generate new AWS access keys in IAM Console
# 2. Update m3admin.conf with new credentials
# 3. Test connection with a simple command
# 4. Delete old AWS access keys in IAM Console

```

Audit Monitoring

```

# Set up automated audit monitoring

# Daily audit report (save to file, then email)
0 9 * * * /usr/local/bin/m3admin environment audit \
  --from_date $(date -d '1 day ago' +%Y-%m-%d) \
  --limit 100 \
  --json > /tmp/daily_audit_$(date +%Y%m%d).json && \
  mail -s "M3 Admin Daily Audit Report - $(date +%Y-%m-%d)" \

```

```
security@example.com < /tmp/daily_audit_$(date +%Y%m%d).json

# Weekly failed operations report (filter with jq)
0 9 * * 1 /usr/local/bin/m3admin environment audit \
  --from_date $(date -d '7 days ago' +%Y-%m-%d) \
  --limit 100 \
  --json | \
  jq '.items[] | select(.Success == false)' > /tmp/failed_ops_$(date +%Y%m%d).json && \
  mail -s "M3 Admin Weekly Failed Operations - $(date +%Y-%m-%d)" \
  ops@example.com < /tmp/failed_ops_$(date +%Y%m%d).json
```

Performance Tuning

RabbitMQ Optimization

```
# m3admin.conf

# Increase timeout for long-running operations
rabbit_timeout=300

# Use dedicated exchanges for better routing
rabbit_exchange=m3_exchange

# Multiple queues for parallel processing
rabbit_request_queue=m3_request_high_priority
rabbit_response_queue=m3_response_${INSTANCE_ID}
```

[Content ↑](#)

8. M3 Admin Integration Details

The following information is useful to enable M3 Admin integration with other tools.

Project Links

- **Source Code:** <https://git.epam.com/epmc-eoos/m3admin/-/tree/master>
- **Documentation:** <https://git.epam.com/epmc-eoos/m3admin/-/blob/develop/README.md>
- **Changelog:** <https://git.epam.com/epmc-eoos/m3admin/-/blob/develop/CHANGELOG.md>
- **Issue Tracker:** <https://git.epam.com/epmc-eoos/m3admin/-/issues>

Related Projects

- **Modular-CLI:** <https://github.com/epam/modular-cli>
- **Modular-SDK:** <https://github.com/epam/modular-sdk>
- **Modular-CLI-SDK:** <https://github.com/epam/modular-cli-sdk>
- **Modular-API:** <https://git.epam.com/epmc-eoos/m3-modular-admin>
- **M3 Server:** <https://git.epam.com/epmc-eoos/m3-backend>

PyPI Packages

- **modular-cli:** <https://pypi.org/project/modular-cli/>
- **modular-sdk:** <https://pypi.org/project/modular-sdk/>
- **modular-cli-sdk:** <https://pypi.org/project/modular-cli-sdk/>

Version Information

- **Current Version:** 4.170.0
- **Supported Python Versions:** 3.14
- **Minimum Modular-SDK Version:** Check `pyproject.toml`
- **License:** Proprietary (EPAM Systems)

Support

Technical Support:

- **Email:** SupportSyndicateTeam@epam.com
- **Team:** EPAM Cloud Orchestration Team (EOOS)

For Issues:

- Report bugs via GitLab Issues
- Include version, mode, and error logs
- Provide steps to reproduce

For Questions:

- Check this documentation first
- Search existing GitLab issues
- Contact support email

Contributing

This is an internal EPAM project. For contribution guidelines, contact the development team.

Deprecation Policy

M3 Admin follows semantic versioning and a three-stage deprecation process:

1. **Deprecation Warning:** Command marked deprecated, warning shown, fully functional
2. **Hidden:** Command hidden from help, warning shown, fully functional
3. **Removal:** Command completely removed

Deprecated commands are typically removed after 2-3 major versions.

Release Schedule

- **Major Releases:** Yearly (breaking changes)
- **Minor Releases:** Quarterly (new features)
- **Patch Releases:** As needed (bug fixes)

Documentation Updates

This documentation is maintained in sync with M3 Admin releases. Last updated: November 2025

For the latest documentation, always refer to the develop branch in GitLab.

[Content](#) ↑

9. Annexes

Annex 1: Common Use Cases

- [Use Case 1: Onboard New AWS Account](#) - Activate a new AWS account for a customer
- [Use Case 2: Multi-Cloud Tenant Setup](#) - Set up a customer across AWS, Azure, and GCP
- [Use Case 3: Quarterly Compliance Audit](#) - Generate compliance audit report for Q1
- [Use Case 4: Credential Rotation](#) - Rotate all application credentials for security
- [Use Case 5: Disaster Recovery](#) - Recover from database corruption or data loss

Use Case 1: Onboard New AWS Account

Objective: Activate a new AWS account for a customer

Steps:

```
# 1. Add customer (if new)
m3admin environment add_customer \
  --name "ACME Corporation" \
  --customer_id ACME_CORP \
  --display_name "ACME Corp"

# 2. Add AWS application credentials
m3admin application add \
  --customer_id ACME_CORP \
  --type AWS_ROLE \
  --description "ACME AWS Management Role" \
  --account_number 123456789012 \
  --role MaestroManagementRole \
  --secret_key_path ./acme_aws_role.json

# 3. Activate AWS tenant
m3admin aws activate \
  --tenant acme-prod-001 \
  --tenant_group production \
  --display_name "ACME Production Account" \
  --tenant_customer ACME_CORP \
  --account_number 123456789012 \
  --activation_date $(date +%Y-%m-%d)

# 4. Add contacts
m3admin tenant add_contacts \
  --tenant acme-prod-001 \
  --contacts "admin@acme.com,ops@acme.com"

# 5. Set quotas
m3admin tenant set_instance_quota \
  --tenant acme-prod-001 \
  --region us-east-1 \
  --instance_type t3.large \
  --quota 100

# 6. Verify activation
m3admin tenant check_status --tenant acme-prod-001
```

[↑ Back to the list](#)

Use Case 2: Multi-Cloud Tenant Setup

Objective: Set up a customer across AWS, Azure, and GCP

Steps:

```
# 1. Add customer
m3admin environment add_customer \
  --name "MultiCloud Inc" \
  --customer_id MULTICLOUD \
  --display_name "MultiCloud Inc"

# 2. Add credentials for each cloud
# AWS
m3admin application add \
  --customer_id MULTICLOUD \
  --type AWS_ROLE \
  --description "MultiCloud AWS" \
  --account_number 123456789012 \
  --role MaestroRole \
  --secret_key_path ./multicloud_aws.json

# Azure
m3admin application add \
  --customer_id MULTICLOUD \
  --type AZURE_CREDENTIALS \
  --description "MultiCloud Azure" \
  --tenant_id "azure-tenant-uuid" \
  --client_id "azure-client-uuid" \
  --secret_key_path ./multicloud_azure.json

# GCP
m3admin application add \
  --customer_id MULTICLOUD \
  --type GOOGLE \
  --description "MultiCloud GCP" \
  --project_id multicloud-gcp-project \
  --secret_key_path ./multicloud_gcp.json

# 3. Activate tenants
m3admin aws activate \
  --tenant multicloud-aws-001 \
  --tenant_group production \
  --display_name "MultiCloud AWS" \
  --tenant_customer MULTICLOUD \
  --account_number 123456789012

m3admin azure activate \
  --tenant multicloud-azure-001 \
  --tenant_group production \
  --display_name "MultiCloud Azure" \
  --tenant_customer MULTICLOUD \
  --subscription_id "azure-subscription-uuid" \
  --tenant_id "azure-tenant-uuid"

m3admin google activate \
  --tenant multicloud-gcp-001 \
  --tenant_group production \
```

```
--display_name "MultiCloud GCP" \  
--tenant_customer MULTICLOUD \  
--project_id multicloud-gcp-project
```

[↑ Back to the list](#)

Use Case 3: Quarterly Compliance Audit

Objective: Generate compliance audit report for Q1

Steps:

1. Export full audit data for Q1 2025

```
m3admin environment audit \  
  --from_date 2025-01-01 \  
  --to_date 2025-03-31 \  
  --limit 100 \  
  --json > /compliance/Q1_2025_full_audit.json
```

Convert to CSV using jq

```
jq -r '["User","Timestamp","Group","Command","Success","Result"],  
      (.items[] | [.User, .Timestamp, .Group, .Command, .Success, .Result]) | @csv' \  
  /compliance/Q1_2025_full_audit.json > /compliance/Q1_2025_full_audit.csv
```

2. Generate failed operations report (filter with jq)

```
m3admin environment audit \  
  --from_date 2025-01-01 \  
  --to_date 2025-03-31 \  
  --limit 100 \  
  --json | \  
jq '.items[] | select(.Success == false)' > /compliance/Q1_2025_failed_ops.json
```

3. Generate privileged operations report for AWS

```
m3admin environment audit \  
  --from_date 2025-01-01 \  
  --to_date 2025-03-31 \  
  --group aws \  
  --command activate \  
  --limit 100 \  
  --json > /compliance/Q1_2025_aws_activate.json
```

```
m3admin environment audit \  
  --from_date 2025-01-01 \  
  --to_date 2025-03-31 \  
  --group aws \  
  --command deactivate \  
  --limit 100 \  
  --json >> /compliance/Q1_2025_aws_deactivate.json
```

Repeat for Azure and Google

```
m3admin environment audit \  
  --from_date 2025-01-01 \  
  --to_date 2025-03-31 \  
  --group azure \  
  --command activate \  
  --limit 100 \  
  --json > /compliance/Q1_2025_azure_activate.json
```

```

m3admin environment audit \
  --from_date 2025-01-01 \
  --to_date 2025-03-31 \
  --group google \
  --command activate \
  --limit 100 \
  --json > /compliance/Q1_2025_google_activate.json

# Merge all privileged operations and convert to CSV
jq -s 'map(.items[]) | unique_by(.Timestamp)' \
  /compliance/Q1_2025_*_activate.json \
  /compliance/Q1_2025_*_deactivate.json | \
jq -r '["User","Timestamp","Group","Command","Success"],
      ([.] | [User, Timestamp, Group, Command, Success]) | @csv' \
  > /compliance/Q1_2025_privileged_ops.csv

# 4. Email comprehensive report
(
  echo "Q1 2025 M3 Admin Compliance Audit Report"
  echo "=====
  echo ""
  echo "Period: 2025-01-01 to 2025-03-31"
  echo ""
  echo "Reports generated:"
  echo "- Full audit: /compliance/Q1_2025_full_audit.csv"
  echo "- Failed operations: /compliance/Q1_2025_failed_ops.json"
  echo "- Privileged operations: /compliance/Q1_2025_privileged_ops.csv"
  echo ""
  echo "Summary:"
  echo "Total events: $(jq '.items | length' /compliance/Q1_2025_full_audit.json)"
  echo "Failed operations: $(jq '. | length' /compliance/Q1_2025_failed_ops.json)"
  echo "Privileged operations: $(wc -l < /compliance/Q1_2025_privileged_ops.csv)"
) | mail -s "Q1 2025 Compliance Audit Report" \
  compliance@example.com,legal@example.com,security@example.com

```

[↑ Back to the list](#)

Use Case 4: Credential Rotation

Objective: Rotate all application credentials for security

Steps:

```

# 1. List all applications
m3admin application describe --customer_id CUSTOMER_001 --json > apps.json

# 2. Rotate each application (extract application IDs with jq)
for app_id in $(jq -r '.items[]?.application_id // .Message[]?.application_id' apps.json); do
  echo "Rotating credentials for: $app_id"
  m3admin application rotate_credentials --application_id $app_id
  sleep 5 # Wait between rotations
done

# 3. Verify rotations in audit
m3admin environment audit \
  --command rotate_credentials \
  --from_date $(date +%Y-%m-%d) \
  --limit 100 \

```



```
--table
```

```
# 4. Send confirmation email (save to file first, then email)
```

```
m3admin environment audit \
  --command rotate_credentials \
  --from_date $(date +%Y-%m-%d) \
  --limit 100 \
  --json > /tmp/rotation_audit_$(date +%Y%m%d).json
```

```
mail -s "Application Credentials Rotated - $(date +%Y-%m-%d)" \
  security@example.com < /tmp/rotation_audit_$(date +%Y%m%d).json
```

[↑ Back to the list](#)

Use Case 5: Disaster Recovery

Objective: Recover from database corruption or data loss

Steps:

```
# 1. Stop M3 Admin operations
```

```
# (Prevent new changes during recovery)
```

```
# 2. Restore from backup (MongoDB example)
```

```
mongorestore \
  --uri="mongodb://admin:password@localhost:27017/maestro" \
  --nsInclude="maestro.*" \
  /backup/m3admin/20250120/mongodb/
```

```
# 3. Restore configuration
```

```
cp /backup/m3admin/20250120/m3admin.conf $M3ADMIN_CONF/
cp /backup/m3admin/20250120/m3admin_policy.json $M3ADMIN_CONF/
```

```
# 4. Verify data integrity
```

```
m3admin environment list_customers
m3admin environment list_tenants
```

```
# 5. Check recent operations from audit
```

```
m3admin environment audit \
  --from_date 2025-01-19 \
  --limit 50 \
  --table
```

```
# 6. Resume normal operations
```

[↑ Back to the list](#)

Annex 2: Troubleshooting Guide

- [Connection Issues](#)
 - Cannot connect to RabbitMQ
 - AWS credentials not working
 - MongoDB connection failed
- [Permission Issues](#)
 - Command execution denied by policy
 - Cannot access audit data
- [Performance Issues](#)
 - Commands running slowly

- Audit queries slow
- **Installation Issues**
 - M3 Admin installation failed
 - Autocompletion not working
- **Configuration Issues**
 - Configuration file not found
 - Policy file invalid

Connection Issues

Problem: Cannot connect to RabbitMQ

Symptoms:

Error: Connection refused to RabbitMQ
Error: Timeout waiting for response

Diagnostics:

```
# Check RabbitMQ connectivity
telnet rabbitmq.example.com 5672

# Check configuration
grep rabbit_connection_url $M3ADMIN_CONF/m3admin.conf

# Test with verbose logging
export M3ADMIN_LOG_LEVEL=DEBUG
m3admin environment list_customers
```

Solutions:

1. Verify RabbitMQ is running: `systemctl status rabbitmq-server`
2. Check network connectivity: `ping rabbitmq.example.com`
3. Verify credentials in connection URL
4. Check firewall rules allow port 5672
5. Verify queue names are correct

Problem: AWS credentials not working

Symptoms:

Error: The security token included in the request is invalid
Error: Access Denied

Diagnostics:

```
# Check AWS credentials
aws sts get-caller-identity

# Check M3 Admin AWS configuration
grep -E "aws_access_key_id|account_id|access_role" $M3ADMIN_CONF/m3admin.conf

# Enable debug logging
export M3ADMIN_LOG_LEVEL=DEBUG
m3admin aws activate --tenant test --help
```

Solutions:

1. Verify AWS credentials are valid

2. Check IAM permissions
3. Verify role trust relationship (if using access_role)
4. Check MFA code is correct (if using MFA)
5. Clear credential cache: `rm $M3ADMIN_CONF/*.json`
6. Regenerate session token

Problem: MongoDB connection failed

Symptoms:

Error: Failed to connect to MongoDB
Error: Authentication failed

Diagnostics:

```
# Test MongoDB connection
mongosh "mongodb://localhost:27017/maestro" --username admin --password

# Check M3 Admin MongoDB configuration
grep mongo_uris $M3ADMIN_CONF/m3admin.conf

# Check network connectivity
telnet mongodb.example.com 27017
```

Solutions:

1. Verify MongoDB is running: `systemctl status mongod`
2. Check MongoDB credentials
3. Verify connection string format
4. Check network connectivity
5. Verify database name exists
6. Check MongoDB authentication settings

[↑ Back to the list](#)

Permission Issues

Problem: Command execution denied by policy

Symptoms:

Error: You don't have permission to execute this command
Error: Access denied by policy

Diagnostics:

```
# Check current policy
m3admin policy

# Verify specific command is allowed
m3admin policy | jq '.[ ] | select(.Group=="aws") | .Resources'

# Check policy file exists
ls -la $M3ADMIN_CONF/m3admin_policy.json
```

Solutions:

1. Update policy file to include required commands
2. Check policy JSON syntax is valid

3. Verify policy file path is correct
4. Restart M3 Admin after policy changes
5. Contact administrator for policy update

Problem: Cannot access audit data

Symptoms:

Error: Access denied to M3adminAudit table
Error: Unable to query audit events

Diagnostics:

```
# Check audit configuration
grep -E "stand_alone_audit|send_audit" $M3ADMIN_CONF/m3admin.conf

# Check database access (DynamoDB)
aws dynamodb describe-table --table-name M3adminAudit

# Check database access (MongoDB)
mongosh --eval "db.M3adminAudit.findOne()" "mongodb://localhost:27017/maestro"
```

Solutions:

1. Verify `stand_alone_audit=True` is set
2. Check database permissions
3. Verify audit table exists
4. Check policy allows `environment audit` command
5. Verify network connectivity to database

[↑ Back to the list](#)

Performance Issues

Problem: Commands running slowly

Symptoms:

Commands take longer than 30 seconds
Timeouts on long-running operations

Diagnostics:

```
# Check RabbitMQ queue lengths
# (Long queues indicate backlog)

# Check database performance
# Monitor DynamoDB consumed capacity
# Monitor MongoDB slow query log

# Enable debug logging to identify bottleneck
export M3ADMIN_LOG_LEVEL=DEBUG
time m3admin tenant describe --tenant test-tenant
```

Solutions:

1. Increase `rabbit_timeout` in configuration
2. Check RabbitMQ server resources (CPU, memory)
3. Scale database (increase DynamoDB capacity or MongoDB resources)

4. Check network latency
5. Optimize database indexes
6. Use `get_operation_status` for async operations

Problem: Audit queries slow

Symptoms:

Audit commands take > 10 seconds
Timeout on large date ranges

Diagnostics:

```
# Check index usage (MongoDB)
mongosh --eval "db.M3adminAudit.getIndexes()" "mongodb://localhost:27017/maestro"

# Check DynamoDB performance metrics
aws cloudwatch get-metric-statistics \
  --namespace AWS/DynamoDB \
  --metric-name ConsumedReadCapacityUnits \
  --dimensions Name=TableName,Value=M3adminAudit \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 \
  --statistics Average
```

Solutions:

1. Create missing indexes (see [Advanced Configuration](#))
2. Reduce query date range
3. Use `--limit` parameter to restrict results
4. Increase DynamoDB read capacity
5. Archive old audit data
6. Use more specific filters (`--user`, `--group`, `--command`)

[↑ Back to the list](#)

Installation Issues

Problem: M3 Admin installation failed

Symptoms:

Error: pip install failed
Error: Module not found

Diagnostics:

```
# Check Python version
python --version
# Should be 3.14

# Check pip version
pip --version

# Check for conflicting packages
pip list | grep modular
```

```
# Try installation with verbose output
pip install --verbose ./src
```

Solutions:

1. Upgrade Python to 3.14
2. Upgrade pip: `pip install --upgrade pip`
3. Use virtual environment
4. Uninstall conflicting packages
5. Install in editable mode: `pip install -e ./src`
6. Check for filesystem permissions

Problem: Autocompletion not working

Symptoms:

Tab completion doesn't work after enabling
No suggestions when pressing TAB

Diagnostics:

```
# Check if M3_ADMIN_HOME is set
echo $M3_ADMIN_HOME

# Check if completion script exists
ls -la /etc/profile.d/*m3admin*
# or
ls -la $M3_ADMIN_HOME/*complete*

# Check shell type
echo $SHELL
```

Solutions:

1. Set `M3_ADMIN_HOME` environment variable
2. Restart shell: `exec $SHELL`
3. Source RC file: `source ~/.bashrc` or `source ~/.zshrc`
4. Re-run: `m3admin enable_autocomplete`
5. Check shell compatibility (Bash 4.0+ or Zsh 5.0+)
6. For system-wide: use `sudo` to enable

[↑ Back to the list](#)

Configuration Issues

Problem: Configuration file not found

Symptoms:

```
Error: Configuration file not found
Error: M3ADMIN_CONF not set
```

Diagnostics:

```
# Check environment variable
echo $M3ADMIN_CONF

# Check file exists
ls -la $M3ADMIN_CONF/m3admin.conf
```

```
# Check file permissions
stat $M3ADMIN_CONF/m3admin.conf
```

Solutions:

1. Set M3ADMIN_CONF environment variable
2. Create configuration file at correct location
3. Check file permissions (should be readable)
4. Verify file path has no typos
5. Add to shell RC file for persistence

Problem: Policy file invalid**Symptoms:**

```
Error: Failed to parse policy file
Error: Invalid policy syntax
```

Diagnostics:

```
# Validate JSON syntax
jq '.' $M3ADMIN_CONF/m3admin_policy.json

# Check file exists and readable
ls -la $M3ADMIN_CONF/m3admin_policy.json

# View policy
cat $M3ADMIN_CONF/m3admin_policy.json
```

Solutions:

1. Validate JSON syntax with jq or online validator
2. Check for missing commas, brackets
3. Ensure all strings in double quotes
4. Verify Effect values are “Allow” or “Deny”
5. Start with simple policy and expand

Common Error Messages

Error Message	Likely Cause	Solution
Tenant not found	Tenant doesn't exist or incorrect name	Verify tenant name with environment list_tenants
Access denied	Policy restriction or invalid credentials	Check policy and credentials
Timeout waiting for response	RabbitMQ or network issue	Check RabbitMQ connectivity and increase timeout
Invalid MFA code	Wrong MFA code or time sync issue	Check system time and try new code
The security token is expired	AWS session expired	Clear cache and re-authenticate
Failed to connect to database	Database connection issue	Check connection string and credentials
Operation ID not found	Operation completed or invalid ID	Check ID and try describe command
Module not found	Missing dependency	Reinstall M3 Admin: pip install --force-reinstall ./src

[↑ Back to the list](#)

Version History

Version	Date	Summary
1.0	July 30, 2026	Initial version created

View Changelog: <https://git.epam.com/epmc-ooos/m3admin/-/blob/develop/CHANGELOG.md>

Documentation generated for: **m3admin, version 4.170**